

Universidad ESAN · Robótica (08079) · Semestre 2026-2

RETO DEL BRAZO 2 (RB-2)

El Turno del Brazo

Cinemática directa y acceso concurrente al JetCobot con ROS 2

Informe técnico · Ítems 1 al 4

Informe de grupo

Sebastian Pedro Aguirre Acosta

Anahi Cortez Chinchay

Ramirez Quevedo Karen Noelia

Vara Vargas Valentino Uziel

Prof. Marks Calderón Niquin — mcalderson@esan.edu.pe

Semana 4 (Fase 1) · Presencial · Máximo 4 integrantes · 6.25 % del PEP

Ítem 1 — Cinemática directa verificada

Deducen la tabla Denavit-Hartenberg del JetCobot e implementan $fk(q)$. La validan contra el robot físico: llevan el brazo a una pose conocida, miden la posición real del efector final y comparan con la predicción. Se declara la predicción antes de medir.

Criterio de aceptación: error de posición ≤ 10 mm en las tres poses de verificación.

1.1 Tabla DH del JetCobot

Para el modelado cinemático del JetCobot se elaboró la tabla de parámetros Denavit-Hartenberg (DH) estándar (Tabla 1.1). Los ángulos $q_1, q_2, q_3, q_4, q_5, q_6$ son los valores articulares que el robot reporta y recibe, expresados en grados. Los desfases (offsets) aplicados a la variable articular θ_i representan la diferencia entre la posición cero de cada articulación del robot y la convención geométrica de la tabla DH.

Tabla 1.1. Parámetros Denavit-Hartenberg (DH) estándar del robot JetCobot.

Articulación	θ_i	d_i (mm)	a_i (mm)	α_i
0-1	θ_1	134.75	0	90°
1-2	$\theta_2 - 90^\circ$	0	-110	0°
2-3	θ_3	0	-96	0°
3-4	$\theta_4 - 90^\circ$	63.4	0	90°
4-5	$\theta_5 + 90^\circ$	75.05	0	-90°
5-6	θ_6	50	0	0°

Cada eslabón se modela mediante su matriz de transformación homogénea $A_i = T_{i-1}^i(q_i)$. La pose del efector final respecto a la base del robot se calcula mediante el producto en cadena de las transformaciones articulares:

$$T_0^6(q) = A_1(q_1) \cdot A_2(q_2) \cdot A_3(q_3) \cdot A_4(q_4) \cdot A_5(q_5) \cdot A_6(q_6)$$

La posición cartesiana predicha del efector final $P_{FK} = [x, y, z]^T$ corresponde a los tres primeros elementos de la cuarta columna de la matriz resultante T_0^6 .

1.2 Verificación experimental de la Cinemática Directa

Para validar la función $fk(q)$ se definieron tres poses conocidas $q^{(1)}, q^{(2)}, q^{(3)}$, y se registró su posición predicha $P_{teórica}$ antes de las pruebas físicas. Posteriormente, cada pose se ejecutó en el JetCobot para obtener la posición real P_{real} y calcular el error cartesiano $e = \|P_{teórica} - P_{real}\|$, verificando que este sea ≤ 10 mm.

1.3 Predicción de la cinemática directa (FK)

A continuación se presenta el script en MATLAB para la predicción de la cinemática directa. La función $fk(q)$ recibe un vector fila $q = [q_1, q_2, q_3, q_4, q_5, q_6]$ con los seis ángulos articulares en grados, en el formato que acepta `send_angles()` en el JetCobot, y entrega la posición cartesiana $[x, y, z]$ del efector final respecto a la base del robot, en milímetros.

```
clc;
clear;
close all;

poses = [
```

```

17 -29 43 13 -23 -53;
-37 15 -21 -32 27 -67;
43 -12 34 27 -19 -47
];

fprintf('prediccion de la fk\n');
for i = 1:size(poses, 1)
q = poses(i,:);
pos = fk(q);
fprintf('\npose %d\n', i);
fprintf('q = [%2f %2f %2f %2f %2f %2f] grados\n', q);
fprintf('x = %2f mm\n', pos(1));
fprintf('y = %2f mm\n', pos(2));
fprintf('z = %2f mm\n', pos(3));
fprintf('[ %2f , %2f , %2f ] mm\n', pos(1), pos(2), pos(3));
end

function pos = fk(q)
d1 = 134.75;
a2 = -110;
a3 = -96;
d4 = 63.4;
d5 = 75.05;
d6 = 50;

theta1 = q(1);
theta2 = q(2) - 90;
theta3 = q(3);
theta4 = q(4) - 90;
theta5 = q(5) + 90;
theta6 = q(6);

a1 = dh(theta1, d1, 0, 90);
a2_matriz = dh(theta2, 0, a2, 0);
a3_matriz = dh(theta3, 0, a3, 0);
a4 = dh(theta4, d4, 0, 90);
a5 = dh(theta5, d5, 0, -90);
a6 = dh(theta6, d6, 0, 0);

t06 = a1 * a2_matriz * a3_matriz * a4 * a5 * a6;
pos = t06(1:3, 4)';
end

function A = dh(theta, d, a, alpha)
A = [cosd(theta), -sind(theta)*cosd(alpha), sind(theta)*sind(alpha), a*cosd(theta);
sind(theta), cosd(theta)*cosd(alpha), -cosd(theta)*sind(alpha), a*sind(theta);
0, sind(alpha), cosd(alpha), d;
0, 0, 0, 1];
end

```

```

1 clc;
2 clear;
3 close all;
4
5 poses = [
6     17 -29 43 13 -23 -53;
7     -37 15 -21 -32 27 -67;
8     43 -12 34 27 -19 -47
9 ];
10
11 fprintf('prediccion de la fk\n');
12 for i = 1:size(poses, 1)
13     q = poses(i,:);
14     pos = fk(q);
15     fprintf('\npose %d\n', i);
16     fprintf('q = [%2f %2f %2f %2f %2f %2f] grados\n', q);
17     fprintf('x = %2f mm\n', pos(1));
18     fprintf('y = %2f mm\n', pos(2));
19     fprintf('z = %2f mm\n', pos(3));
20     fprintf('[ %2f , %2f , %2f ] mm\n', pos(1), pos(2), pos(3));
21 end
22
23 function pos = fk(q)
24     d1 = 134.75;
25     a2 = -110;
26     a3 = -96;
27     d4 = 63.4;
28     d5 = 75.05;
29     d6 = 50;
30
31     theta1 = q(1);
32     theta2 = q(2) - 90;
33     theta3 = q(3);
34     theta4 = q(4) - 90;
35     theta5 = q(5) + 90;
36     theta6 = q(6);
37
38     a1 = dh(theta1, d1, 0, 90);
39     a2_matriz = dh(theta2, 0, a2, 0);
40     a3_matriz = dh(theta3, 0, a3, 0);
41     a4 = dh(theta4, d4, 0, 90);
42     a5 = dh(theta5, d5, 0, -90);
43     a6 = dh(theta6, d6, 0, 0);
44
45     t06 = a1 * a2_matriz * a3_matriz * a4 * a5 * a6;
46     pos = t06(1:3, 4)';
47 end
48
49 function A = dh(theta, d, a, alpha)
50     A = [cosd(theta), -sind(theta)*cosd(alpha), sind(theta)*sind(alpha), a*cosd(theta);
51         sind(theta), cosd(theta)*cosd(alpha), -cosd(theta)*sind(alpha), a*sind(theta);
52         0, sind(alpha), cosd(alpha), d;
53         0, 0, 0, 1];
54 end

```

```

prediccion de la fk
pose 1
q = [17.00 -29.00 43.00 13.00 -23.00 -53.00] grados
x = 59.67 mm
y = -68.48 mm
z = 411.87 mm
[ 59.67 , -68.48 , 411.87 ] mm
pose 2
q = [-37.00 15.00 -21.00 -32.00 27.00 -67.00] grados
x = 25.72 mm
y = -70.34 mm
z = 368.19 mm
[ 25.72 , -70.34 , 368.19 ] mm
pose 3
q = [43.00 -12.00 34.00 27.00 -19.00 -47.00] grados
x = 26.02 mm
y = -84.68 mm
z = 416.27 mm
[ 26.02 , -84.68 , 416.27 ] mm

```

Figura 1.1. Resultados de la predicción de posición cartesiana (FK) en la consola de MATLAB para las tres poses evaluadas.

1.4 Predicciones previas

Las posiciones predichas de la Tabla 1.2 se obtuvieron con la función $fk(q)$ en MATLAB, a partir de la tabla DH y de los ángulos articulares seleccionados, sin emplear lecturas del robot como entrada del modelo.

Tabla 1.2. Posiciones cartesianas predichas por el modelo de cinemática directa (FK)

Pose	x predicho (mm)	y predicho (mm)	z predicho (mm)
1	59.67	-68.48	411.87
2	25.72	-70.34	368.19
3	26.02	-84.68	416.27

1.5 Medición en el JetCobot

A continuación se presenta el script en Python ejecutado en el JetCobot. Cada una de las tres poses $q = [q_1, q_2, q_3, q_4, q_5, q_6]$ se envía en grados con `send_angles()`; tras una espera de 5 s, para que el brazo se estabilice, se lee con `get_coords()` la posición cartesiana real $[x, y, z]$ en milímetros.

```
import time

poses = [
    [17, -29, 43, 13, -23, -53],
    [-37, 15, -21, -32, 27, -67],
    [43, -12, 34, 27, -19, -47]
]

for i, pose in enumerate(poses, 1):

    print(f"\npose {i}")
    print("angulos:", pose)

    mc.send_angles(pose, 10)

    time.sleep(5)

    coords = mc.get_coords()

    print("posicion del JetCobot:")
    print(f"x = {coords[0]:.2f} mm")
    print(f"y = {coords[1]:.2f} mm")
    print(f"z = {coords[2]:.2f} mm")
    print(f"[ {coords[0]:.2f} , {coords[1]:.2f} , {coords[2]:.2f} ] mm")

print("\ntodas las poses fueron llevadas a cabo")
```

```

print(f"z = {coords[2]:.2f} mm")
print(f"[ {coords[0]:.2f} , {coords[1]:.2f} , {coord")

print("\ntodas las poses fueron llevadas a cabo")

pose 1
angulos: [17, -29, 43, 13, -23, -53]
posicion del JetCobot:
x = 60.70 mm
y = -68.60 mm
z = 406.60 mm
[ 60.70 , -68.60 , 406.60 ] mm

pose 2
angulos: [-37, 15, -21, -32, 27, -67]
posicion del JetCobot:
x = 25.50 mm
y = -72.20 mm
z = 362.30 mm
[ 25.50 , -72.20 , 362.30 ] mm

pose 3
angulos: [43, -12, 34, 27, -19, -47]
posicion del JetCobot:
x = 21.50 mm
y = -78.60 mm
z = 416.50 mm
[ 21.50 , -78.60 , 416.50 ] mm

todas las poses fueron llevadas a cabo

[ ]:

```

Figura 1.2. Lecturas de posición cartesiana real medidas en el JetCobot mediante `get_coords()` para las tres poses ejecutadas.

1.6 Comparación: Predicción teórica vs. Medición real

La referencia para evaluar el modelo son las posiciones que reporta el robot mediante `get_coords()`. El error de posición se calcula como la distancia euclidiana entre la predicción teórica (P_{FK}) y la medición del robot (P_{robot}), y se contrasta con el criterio de aceptación ($e \leq 10$ mm).

Para esta verificación se desarrolló un script en MATLAB que procesa las posiciones predichas y las lecturas del JetCobot, y calcula el error por eje, el error promedio y la distancia euclidiana:

```

clc;
clear;
close all;

poses = [
17 -29 43 13 -23 -53;
-37 15 -21 -32 27 -67;
43 -12 34 27 -19 -47
];

reales = [
60.70 -68.60 406.60;
25.50 -72.20 362.30;
21.50 -78.60 416.50
];

fprintf('comparacion con el jetcobot\n');
for i = 1:size(poses, 1)
pos = fk(poses(i,:));

error_x = abs(pos(1) - reales(i,1));
error_y = abs(pos(2) - reales(i,2));
error_z = abs(pos(3) - reales(i,3));

```

```

error_promedio = (error_x + error_y + error_z) / 3;
error_euclidiano = norm(pos - reales(i,:));

if error_euclidiano <= 10
    estado = 'CUMPLE';
else
    estado = 'NO CUMPLE';
end

fprintf('pose %d\n', i);
fprintf('error x = %.2f mm\n', error_x);
fprintf('error y = %.2f mm\n', error_y);
fprintf('error z = %.2f mm\n', error_z);
fprintf('error promedio = %.2f mm\n', error_promedio);
fprintf('error euclidiano = %.2f mm (criterio <= 10 mm: %s)\n', ...
    error_euclidiano, estado);
end

function pos = fk(q)
d1 = 134.75;
a2 = -110;
a3 = -96;
d4 = 63.4;
d5 = 75.05;
d6 = 50;

theta1 = q(1);
theta2 = q(2) - 90;
theta3 = q(3);
theta4 = q(4) - 90;
theta5 = q(5) + 90;
theta6 = q(6);

a1 = dh(theta1, d1, 0, 90);
a2_matriz = dh(theta2, 0, a2, 0);
a3_matriz = dh(theta3, 0, a3, 0);
a4 = dh(theta4, d4, 0, 90);
a5 = dh(theta5, d5, 0, -90);
a6 = dh(theta6, d6, 0, 0);

t06 = a1 * a2_matriz * a3_matriz * a4 * a5 * a6;
pos = t06(1:3, 4)';
end

function A = dh(theta, d, a, alpha)
A = [cosd(theta), -sind(theta)*cosd(alpha), sind(theta)*sind(alpha), a*cosd(theta);
    sind(theta), cosd(theta)*cosd(alpha), -cosd(theta)*sind(alpha), a*sind(theta);
    0, sind(alpha), cosd(alpha), d;
    0, 0, 0, 1];
end

```

```

1  clc;
2  clear;
3  close all;
4
5  poses = [
6      17 -29 43 13 -23 -53;
7      -37 15 -21 -32 27 -67;
8      43 -12 34 27 -19 -47
9  ];
10
11  reales = [
12      60.70 -68.60 406.60;
13      25.50 -72.20 362.30;
14      21.50 -78.60 416.50
15  ];
16
17  fprintf('comparacion con el jetcobot\n');
18  for i = 1:size(poses, 1)
19      pos = fk(poses(i,:));
20
21      error_x = abs(pos(1) - reales(i,1));
22      error_y = abs(pos(2) - reales(i,2));
23      error_z = abs(pos(3) - reales(i,3));
24
25      error_promedio = (error_x + error_y + error_z) / 3;
26      error_euclidiano = norm(pos - reales(i,:));
27
28      if error_euclidiano <= 10
29          estado = 'CUMPLE';
30      else
31          estado = 'NO CUMPLE';
32      end
33
34      fprintf('pose %d\n', i);
35      fprintf('error x = %.2f mm\n', error_x);
36      fprintf('error y = %.2f mm\n', error_y);
37      fprintf('error z = %.2f mm\n', error_z);
38      fprintf('error promedio = %.2f mm\n', error_promedio);
39      fprintf('error euclidiano = %.2f mm (criterio <= 10 mm: %s)\n', error_euclidiano, estado);

```

Figura 1.3. Resultados del cálculo de error de posición cartesiana y verificación del criterio de aceptación en la consola de MATLAB.

Tabla 1.3. Comparación entre la predicción teórica (FK) y la medición física en el JetCobot.

Pose	Predicción FK [x,y,z] (mm)	Medición robot [x,y,z] (mm)	Error por eje $ \Delta x , \Delta y , \Delta z $ (mm)	Error euclidiano e (mm)	Criterio (≤ 10 mm)
1	[59.67, -68.48, 411.87]	[60.70, -68.60, 406.60]	[1.03, 0.12, 5.27]	5.37	CUMPLE
2	[25.72, -70.34, 368.19]	[25.50, -72.20, 362.30]	[0.22, 1.86, 5.89]	6.18	CUMPLE
3	[26.02, -84.68, 416.27]	[21.50, -78.60, 416.50]	[4.52, 6.08, 0.23]	7.58	CUMPLE

1.7 Análisis de resultados

Los errores de posición euclidianos (e) en las tres poses fueron de 5.37 mm, 6.18 mm y 7.58 mm, valores que cumplen el criterio de aceptación ($e \leq 10$ mm) con un margen mínimo de 2.42 mm en la pose 3. Por eje, los errores máximos fueron de 4.52 mm en x, 6.08 mm en y y 5.89 mm en z, todos inferiores a 10 mm. El error en z de las poses 1 y 2 (5.27 y 5.89 mm) presenta un sesgo que se analiza en la sección 1.8.

1.8 Análisis del error

Alcance de la referencia

La posición de referencia proviene de `get_coords()`, es decir, del cálculo interno del firmware del JetCobot a partir de sus propias lecturas articulares. Por lo tanto, esta validación contrasta el modelo DH del equipo con el modelo cinemático del fabricante, y no con una medición externa del efector (por ejemplo, con regla o cinta). Se deja constancia de ello para interpretar los resultados dentro de su alcance.

Comportamiento del error

- **Plano XY:** los errores en x e y estuvieron entre 0.12 y 6.08 mm, lo que sugiere que la geometría horizontal del modelo es coherente con la del robot.
- **Eje z (poses 1 y 2):** la FK predice una altura mayor que la medida (5.27 mm y 5.89 mm, con el mismo signo). En la pose 3 la diferencia es de solo 0.23 mm, por lo que el sesgo no es constante entre configuraciones.

Posibles causas

Como el error en z no aparece en las tres poses, es poco probable que provenga de un único parámetro mal estimado de la tabla DH (un d_1 incorrecto, por ejemplo, desplazaría z por igual en todas). Las causas más plausibles, que no se verificaron experimentalmente, son las siguientes:

- Diferencias entre las dimensiones nominales de la tabla DH (d_i , a_i) y las que usa el firmware, o un punto de referencia distinto del efector (centro de la brida frente al punto de la herramienta).
- Diferencia entre el ángulo comandado y el realmente alcanzado por cada servo (holgura, carga por gravedad), que afecta más a las poses con el brazo extendido.
- Resolución y redondeo de las lecturas de `get_coords()`.

Mejora propuesta

Se proponen tres acciones para reducir el error residual:

1. **Lectura articular real con `get_angles()`:** reemplazar los ángulos comandados por los que devuelve `get_angles()` una vez estabilizado el movimiento. De este modo se separa el efecto de la tabla DH del de las desviaciones mecánicas de los servos (holguras, par insuficiente o flexión por gravedad).
2. **Ajuste del vector de herramienta (d_6 o TCP):** modificar d_6 para que coincida con la distancia al Tool Center Point (TCP) que define el firmware, de modo que se elimine la diferencia entre el centro de la brida y el origen del efector final.
3. **Revisión del cero articular (offset):** verificar la calibración de los ceros de las articulaciones 2, 4 y 5. Como el error en z varía con la postura, un pequeño desfase en el cero de las articulaciones intermedias podría amplificarse en posturas extendidas por efecto del brazo de palanca.

Dado que las tres poses cumplen el criterio (≤ 10 mm), estas mejoras no son indispensables para dar por válida la verificación actual.

1.9 Conclusión

Se dedujo la tabla de parámetros DH del JetCobot y se implementó la función $fk(q)$ para su cinemática directa. En las tres poses de verificación el error de posición fue de 5.37 mm, 6.18 mm y 7.58 mm, dentro del criterio de aceptación (≤ 10 mm). La función queda validada frente a la referencia que reporta el firmware y lista para usarse como validador de admisión en el `armBroker` (Ítem 2) y como instrumento de auditoría de la cinemática inversa del firmware (Ítem 4).

Ítem 2 — El broker con cola de prioridades

El Ítem 2 construye el nodo `arm_broker`: un servidor de acciones de ROS 2 que convierte el acceso concurrente de cuatro clientes en un servicio ordenado sobre un recurso exclusivo, el JetCobot. Sus tres garantías son la **exclusión mutua** (a lo sumo una meta en ejecución y un único publicador en `/joint_states`), la **admisión validada con la cinemática directa** del Ítem 1 y una **política de cola** con envejecimiento (aging) que evita la inanición.

2.1 Asignación de autoría y responsabilidades técnicas

El broker se dividió en cuatro módulos y cada uno es responsable técnico de una capa verificable del sistema (Tabla 2.1). La asignación coincide con las cuatro partes del script de parches `aplicar_parches.py` (parte 1 a parte 4), de modo que cada bloque de código tiene a uno de los miembros como autor.

Tabla 2.1. Responsabilidades técnicas por integrante.

Integrante	Módulo / función	Responsabilidad técnica	Evidencia
Sebastian Pedro Aguirre Acosta	<code>fk.py</code> (parche 1)	Desarrollo e inyección del módulo de cinemática directa: tabla DH, matriz de transformación homogénea, cálculo vectorial de la posición (x, y, z) del efector final y validación geométrica del workspace ($z \leq 0$).	Figura 2.6; Tabla 2.8; Ítem 1
Anahi Cortez Chinchay	<code>politicas.py</code> (parche 2)	Capa de estrategias y políticas de encolado: regla FIFO y prioridad dinámica con envejecimiento (aging, $\tau = 8.0$ s).	Tablas 2.9, 3.3 y 3.4; Figura 3.3
Ramirez Quevedo Karen Noelia	<code>broker.py</code> : <code>goal_callback</code> , <code>handle_accepted_callback</code> (parche 3)	Filtro de admisión asíncrono y control de concurrencia: límites articulares, espacio de trabajo $z \leq 0$, paso articular máximo y capacidad de cola; contadores y encolado bajo exclusión mutua.	Figura 2.7; Tablas 2.7 y 2.8
Vara Vargas Valentino Uziel	<code>broker.py</code> : <code>hilo worker</code> y <code>execute_callback</code> (parche 4)	Hilo trabajador único (worker exclusivo), bucle de ejecución multihilo con exclusión mutua, publicación de feedback durante el movimiento y control de cancelaciones y abortos.	Figuras 2.11 y 2.12; Tablas 2.6 y 2.10

2.2 Arquitectura: el brazo como recurso exclusivo

El JetCobot es un recurso no compartible. Si varios clientes publicaran directamente en `/joint_states`, el robot obedecería mensajes intercalados y se movería a poses que nadie solicitó: un fallo silencioso. Por ello solo el nodo `arm_broker`, ejecutado en la Jetson, publica en ese tópico.

El diseño separa dos fases de costo muy distinto. La admisión es barata e inmediata: evaluación de la FK, comparaciones y una inserción en la lista de pendientes. La ejecución es lenta (≈ 3.0 s por meta) y estrictamente serializada. Encolar no es ejecutar: `goal_callback` acepta o rechaza, `handle_accepted_callback` encola y un único worker extrae la meta de mayor prioridad efectiva y la ejecuta. El flujo tiene tres fases: solicitud (Cliente $\rightarrow$ Broker), encolamiento con monitoreo (publicación de `/arm/queue_state` a 5 Hz con identificadores, prioridades y tiempos de espera) y despacho con feedback (Broker $\rightarrow$ brazo y cliente, hasta el estado `SUCCEEDED`).

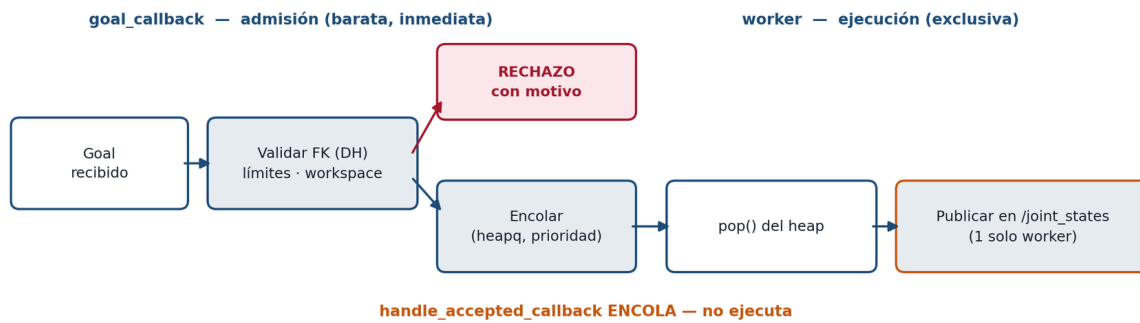


Figura 2.1. Diagrama de secuencia del camino de un goal: admisión, encolado y ejecución exclusiva.

2.3 Entorno de red y despliegue distribuido

Para aislar el tráfico DDS del equipo y comunicar la Raspberry Pi (alumno01@rpi-02) con la Jetson del Brazo 8 se configuró un dominio propio (`ROS_DOMAIN_ID = 102`) y se sustituyó el descubrimiento por multicast por un FastDDS Discovery Server. Las variables se persistieron en `~/bashrc` y se reinició el demonio de ROS 2 (Figura 2.2).

El perfil `super_client_configuration_file.xml` se descargó de la Jetson por SCP y su dirección de loopback (127.0.0.1) se reescribió a la IP real 172.51.9.22 mediante `sed`. Tras exportar `FASTRTPS_DEFAULT_PROFILES_FILE` y reiniciar el demonio, el comando `ros2 node list` mostró los nodos `/arm_broker` y `/control_sync_plan`, lo que confirma la sincronización del grafo entre ambas placas (Figura 2.3). Las pruebas funcionales previas se ejecutaron en un dominio local aislado (99, solo loopback) con el Discovery Server desactivado, para descartar interferencias de red.

Tabla 2.2. Parámetros de red según el tipo de ensayo.

Parámetro	Ensayos en hardware	Pruebas locales aisladas	Función
Identificador de brazo	Brazo 8	—	Hardware físico asignado
<code>ROS_DOMAIN_ID</code>	102	99	Aislamiento del dominio DDS
<code>ROS_LOCALHOST_ONLY</code>	0	1	Descubrimiento en la red física o solo en loopback
<code>RMW_IMPLEMENTATION</code>	<code>rmw_fastrtps_cpp</code>	<code>rmw_fastrtps_cpp</code>	Implementación de middleware DDS
Descubrimiento	Discovery Server 172.51.9.22:11811	Desactivado	Registro centralizado de participantes
Ubicación de <code>arm_broker</code>	Jetson (172.51.9.22)	Raspberry Pi (rpi-02)	Nodo servidor

```

alumno01@rpi-02: ~
alumno01@rpi-02: ~ 80x24
alumno01@rpi-02:~$ echo 'source /opt/ros/humble/setup.bash' >> ~/.bashrc
alumno01@rpi-02:~$ echo 'export ROS_DOMAIN_ID=102' >> ~/.bashrc
alumno01@rpi-02:~$ echo 'export ROS_LOCALHOST_ONLY=0' >> ~/.bashrc
alumno01@rpi-02:~$ echo 'export JETSON=172.51.9.22' >> ~/.bashrc
alumno01@rpi-02:~$ exec bash
alumno01@rpi-02:~$ ros2 daemon stop; ros2 daemon start
The daemon is not running
The daemon has been started
alumno01@rpi-02:~$

```

Figura 2.2. Persistencia de las variables de entorno ROS 2 en ~/.bashrc y reinicio del demonio (ros2 daemon stop / start) en alumno01@rpi-02.

```

alumno01@rpi-02: ~
alumno01@rpi-02: ~ 80x24
alumno01@rpi-02:~$ env | grep -E "ROS|RMW|FASTRTPS"
ros2 daemon stop; ros2 daemon start
ros2 node list
ROS_VERSION=2
ROS_PYTHON_VERSION=3
FASTRTPS_DEFAULT_PROFILES_FILE=/home/alumno01/super_client_configuration_file.xml
ROS_DOMAIN_ID=102
ROS_LOCALHOST_ONLY=0
ROS_DISTRO=humble
ROS_DISCOVERY_SERVER=172.51.9.22:11811
RMW_IMPLEMENTATION=rmw_fastrtps_cpp
The daemon has been stopped
The daemon has been started
WARNING: Be aware that there are nodes in the graph that share an exact name, wh
alumno01@rpi-02:~$

```

Figura 2.3. Configuración del FastDDS Discovery Server en la Raspberry Pi, corrección de direccionamiento IP hacia la Jetson (172.51.9.22) y verificación de descubrimiento de nodos distribuidos (/arm_breaker y /control_sync_plan).

2.4 Kit base, parches de lógica y compilación del workspace

El kit kit_reto_alumno.zip se descomprimió y sus paquetes se copiaron a la carpeta src del workspace ros2_ws en rpi-02. Se compilaron con colcon los dos paquetes del proyecto, arm_breaker_interfaces y arm_breaker, y se verificó con la CLI de ROS 2 la estructura de la acción MoveArm (Figura 2.4). Las advertencias de «Clock skew detected», causadas por el desfase del reloj de la Raspberry Pi respecto a las marcas de tiempo de los archivos, se eliminaron sincronizando la hora por NTP.

Tabla 2.3. Componentes del kit base y herramientas del equipo.

Archivo	Función técnica
arm_breaker/broker.py	Nodo servidor principal: grupos de callbacks, admisión, encolado, worker y publicación de /arm/queue_state.
arm_breaker/fk.py	Cinemática directa y validaciones geométricas para el filtro de admisión.
arm_breaker/politicas.py	Políticas de cola: FIFO y prioridad con envejecimiento.

Archivo	Función técnica
aplicar_parches.py	Inyección idempotente de los cuatro bloques de lógica en fk.py, politicas.py y broker.py.
exportar_csv.py	Lectura de un bag de rosbag2 (almacenamiento sqlite3, .db3) y exportación a queue_state.csv y joint_states.csv.
metricas.py	Espera media y P95 por prioridad, índice de inanición, equidad de Jain y figura comparativa.
predecir_p95.py	Simulación previa de la espera por política, usando politicas.py real (sección 3.4).
verificar_exclusion.py	Auditoría externa de la exclusión mutua sobre los CSV exportados (sección 2.11).

El script aplicar_parches.py actualizó tres módulos mediante cuatro bloques. Es repetible y a prueba de errores: antes de reemplazar, comprueba que el bloque esperado aparezca exactamente una vez; si ya fue aplicado lo deja como estaba y, si no lo encuentra, cancela sin modificar el archivo (Tabla 2.4).

Tabla 2.4. Bloques de lógica inyectados por aplicar_parches.py.

Parche	Archivo	Cambio introducido
1	fk.py	Lista DH con (α , a, d, offset) en radianes según la Tabla 1.1, matriz homogénea de la convención estándar y fk(q) que devuelve la cuarta columna de T_0^6 como (x, y, z).
2	politicas.py	FIFO: elige la meta de menor $t_{llegada}$. Prioridad: puntaje = prioridad + espera/T con $T = 8.0$ s y desempate por llegada más antigua.
3	broker.py	goal_callback: validación en cascada (límite articular, workspace, paso articular, capacidad) con motivo en el log. handle_accepted_callback: encolado bajo cerrojo.
4	broker.py	Hilo worker único (desencola y ejecuta de a uno) y execute_callback con feedback por paso, cancelación en cola y en ejecución, y aborto ante excepción.

La validez del parche se comprobó contando las apariciones de NotImplementedError en broker.py: el resultado fue 0 en el workspace parchado. La recompilación con la opción --symlink-install terminó con 2 paquetes finalizados y la evaluación de FK(q) arrojó una posición de (50.0; -63.4; 415.8) mm, con $z > 0$ conforme a la restricción de seguridad (Figuras 2.5 y 2.6).

```

alumno01@rpi-02: ~/Desktop/ros2_ws
alumno01@rpi-02: ~/Desktop/ros2_ws 80x23
Finished <<< arm_broker_interfaces [3.99s]
Starting >>> arm_broker
Finished <<< arm_broker [5.19s]

Summary: 2 packages finished [12.4s]
1 package had stderr output: arm_broker_interfaces
alumno01@rpi-02:~/Desktop/ros2_ws$ source install/setup.bash
alumno01@rpi-02:~/Desktop/ros2_ws$ ros2 interface show arm_broker_interfaces/action/MoveArm
# priority: 0..255, MAYOR = MAS URGENTE
float64[] joint_positions
string client_id
uint8 priority
---
bool success
string message
float64 wait_time_s
float64 exec_time_s
---
string state
int32 queue_position
float64 elapsed_s
alumno01@rpi-02:~/Desktop/ros2_ws$

```

Figura 2.4. Compilación del workspace con colcon (2 packages finished) e inspección de la acción arm_broker_interfaces/action/MoveArm con sus secciones Goal, Result y Feedback.

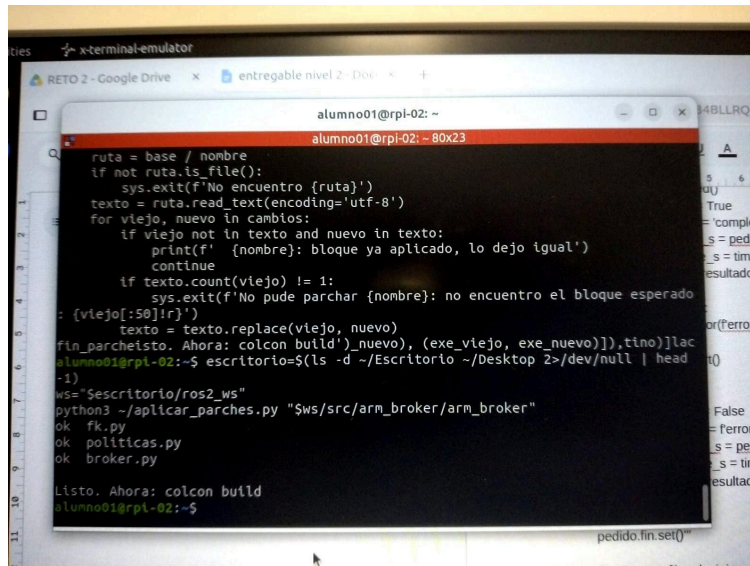


Figura 2.5. Aplicación de aplicar_parches.py sobre fk.py, políticas.py y broker.py, previa a la recompilación del workspace.

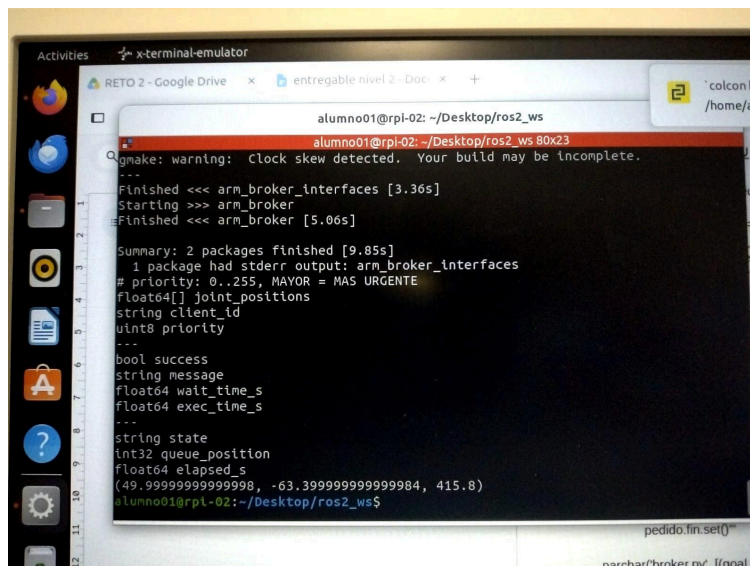


Figura 2.6. Recompilación de arm_broker_interfaces y evaluación de FK(q) con altura $z = 415.8$ mm ($z > 0$).

2.5 Interfaz de acción MoveArm

Tabla 2.5. Estructura de la acción MoveArm y del mensaje de estado QueueState.

Bloque	Campo	Tipo	Descripción
Goal	joint_positions	float64[]	Ángulos articulares objetivo (rad)
Goal	client_id	string	Identificador único del cliente
Goal	priority	uint8	Prioridad de 0 a 255; mayor valor, mayor urgencia
Result	success	bool	Éxito de la operación
Result	message	string	Diagnóstico de la ejecución
Result	wait_time_s	float64	Permanencia en cola antes de ser atendida
Result	exec_time_s	float64	Tiempo de ejecución del movimiento
Feedback	state	string	Estado actual (EXECUTING durante el movimiento)

Bloque	Campo	Tipo	Descripción
Feedback	queue_position	int32	Posición en la cola (0 mientras la meta se ejecuta)
Feedback	elapsed_s	float64	Tiempo transcurrido desde el inicio de la ejecución
QueueState	executing_client, executing_goal_id, executing_elapsed_s	string, string, float	Meta en ejecución (un único identificador por mensaje)
QueueState	queue_length, queued_goal_ids, queued_clients, queued_priorities, queued_wait_s	int, arreglos	Estado de la cola en el instante de publicación
QueueState	total_accepted, total_rejected, total_completed	contadores	Contadores acumulativos publicados a 5 Hz

Se eligió la interfaz de acciones y no un servicio por tres razones. Primero, permite la cancelación explícita mediante `/move_arm/_action/cancel_goal`, de modo que un movimiento peligroso se detiene sin reiniciar el broker. Segundo, es asíncrona: el cliente no bloquea su hilo y recibe feedback periódico. Tercero, la publicación de `/arm/queue_state` a 5 Hz sobre el middleware DDS consume un ancho de banda inferior a 2.5 KB/s, despreciable para la red local.

2.6 Concurrencia y exclusión mutua

El ejecutor multihilo de ROS 2 atiende en paralelo los callbacks de admisión, el temporizador de telemetría y el hilo worker. El estado compartido del broker (la lista de pendientes, el diccionario por `goal_id`, la referencia a la meta en ejecución, los contadores y la pose actual `q_actual`) puede, por tanto, ser leído y escrito desde hilos POSIX distintos. Sin protección existiría una condición de carrera de tipo *comprobar-y-actuar*: por ejemplo, dos callbacks verificarían simultáneamente que el brazo está libre y ambos intentarían tomarlo. El intérprete de Python garantiza la atomicidad de instrucciones individuales, pero no la de secuencias compuestas, por lo que se requiere exclusión mutua explícita. La protección se organiza en tres capas independientes.

Capa 1 — Separación de grupos de callbacks

La arquitectura de dos `CallbackGroup` separa lo que debe ser concurrente de lo que debe ser exclusivo (Tabla 2.6).

Tabla 2.6. Grupos de callbacks y función de cada uno.

Grupo	Callbacks	Tipo	Justificación
Admisión	<code>goal_callback</code> , <code>handle_accepted_callback</code> , <code>cancel_callback</code>	ReentrantCallbackGroup	Permite validar y encolar varios goals en paralelo mientras el brazo ejecuta otro. La admisión es corta (FK, comparaciones e inserción en lista) y no debe esperar al movimiento: un cliente nunca queda bloqueado porque el brazo esté ocupado.
Ejecución	<code>execute_callback</code> (movimiento y feedback)	MutuallyExclusiveCallbackGroup	El ejecutor garantiza que, a lo sumo, un callback del grupo esté activo. Así, un único hilo toma el control del simulador o hardware y solo él invoca la publicación en <code>/joint_states</code> .
Telemetría	Temporizador de <code>/arm/queue_state</code> (5 Hz)	Temporizador periódico	Publica una instantánea de la cola tomada bajo el cerrojo; como la sección crítica es breve, un movimiento de ≈ 3 s no retrasa la publicación del estado.

Capa 2 — Worker único y cerrojo (threading.Lock)

Un único hilo worker ejecuta el bucle de despacho. En cada iteración adquiere el cerrojo `self.lock` y, dentro de la misma sección crítica, comprueba que no haya meta en ejecución (`self.ejecutando` is `None`), elige la siguiente según la política, la extrae de la lista y la marca como ejecutándose. La comprobación y la marca son, por tanto, atómicas: es imposible que dos hilos observen el brazo libre al mismo tiempo. El cerrojo se libera antes de invocar `goal_handle.execute()`; el worker espera luego un evento (`pedido.fin`) que `execute_callback` activa al terminar, y solo entonces vuelve a tomar el cerrojo para limpiar la referencia, actualizar `total_completed` y notificar a la política (atendido).

Esta disciplina tiene dos consecuencias de ingeniería. (i) La sección crítica es breve ($n \leq 20$ elementos) y no incluye el movimiento, de modo que la admisión nunca se bloquea detrás del brazo. (ii) Existe un único cerrojo y ningún camino lo adquiere mientras espera otro recurso, así que no hay espera circular: se elimina una de las cuatro condiciones de Coffman y se descarta el interbloqueo.

Capa 3 — Auditoría externa

La exclusión se verifica con herramientas independientes de la implementación: el conteo de publicadores de `/joint_states` (Figura 2.11) y el script `verificar_exclusion.py` sobre los CSV exportados del bag (sección 2.11). La invariante que se audita es: en todo instante, a lo sumo una meta está en estado `EXECUTING` y solo su ejecución publica en `/joint_states`.

Alcance y límite conocido. `goal_callback` comprueba la capacidad de la cola (`cola_max = 20`) bajo el cerrojo, pero el encolado efectivo ocurre después, en `handle_accepted_callback`, en otra sección crítica. Ante una ráfaga cercana al límite, la cola podría superar `cola_max` en, a lo sumo, el número de clientes concurrentes (4). Esto no afecta la exclusión sobre el brazo; se declara por transparencia y se resolvería moviendo la comprobación a la sección crítica del encolado.

2.7 Admisión validada con FK y rechazos razonados

En `goal_callback`, la cinemática directa del Ítem 1 actúa como portero. Los criterios se evalúan en cascada y el primero que falla produce el rechazo (Tabla 2.7). El protocolo de acciones de ROS 2 solo transmite `ACCEPT` o `REJECT`, sin campo de motivo; por eso la causa legible se registra en el log del broker con el formato `RECHAZADO [cliente]: motivo` y cada rechazo incrementa el contador `total_rejected` publicado en `/arm/queue_state`.

Tabla 2.7. Criterios de admisión, en el orden de evaluación del broker.

Orden	Criterio	Condición de rechazo	Motivo registrado en el log
1	Límite articular	Alguna q_i fuera de su rango mecánico (software limits en <code>fk.py</code>)	límite articular: <detalle>
2	Workspace	z de <code>FK(q)</code> ≤ 0 mm: la herramienta colisionaría con la mesa	fuera del workspace: <detalle>
3	Paso articular máximo	Salto articular respecto a la pose actual q_{actual} superior a <code>paso_max</code>	paso articular excesivo: X rad, maximo Y
4	Capacidad	Longitud de la cola $\geq$ <code>cola_max = 20</code>	cola llena (maximo 20)

Los tres casos de prueba por CLI (Figura 2.7) se evaluaron con la FK propia para asociar cada rechazo con su criterio (Tabla 2.8). Las posiciones cartesianas se calcularon con la misma tabla DH validada en el Ítem 1.

Tabla 2.8. Casos de rechazo y criterio asociado.

Caso	Meta enviada (rad)	Equivalente articular y FK(q)	Resultado	Criterio asociado
1	[3.5, 0, 0, 0, 0, 0]	$q_1 \approx 200.5^\circ$	Goal was rejected	Límite articular de la articulación 1
2	[0, -2.3, -2.3, 1.2, 0, 0]	$q_2 = q_3 \approx -131.8^\circ$, $q_4 \approx 68.8^\circ$ FK = (-80.9; -63.4; -9.1) mm	Goal was rejected	Workspace: $z = -9.1 \text{ mm} \leq 0$
3	[1.5, 0, 0, 0, 0, 0]	$q_1 \approx 85.9^\circ$ FK = (66.8; 45.4; 415.8) mm, dentro del workspace	Goal was rejected	Paso articular máximo (salto de 1.5 rad $\approx 85.9^\circ$ en q_1 desde la pose de reset)

El caso 3 es cinemáticamente válido en posición y límites, y con un único cliente la cola no puede alcanzar 20 elementos; por tanto, su rechazo corresponde al criterio de paso articular y no a una colisión ni a saturación.

```

alumno01@rpi-02: ~
alumno01@rpi-02: ~ 80x24
[INFO] [1790284345.309015836] [arm_client]: [tu_nombre] EXECUTING pos=0 t=2.4s
[INFO] [1790284345.920554336] [arm_client]: [tu_nombre] pose 1: success=True esp
era=0.00s ejec=3.02s total=3.05s - completado
alumno01@rpi-02:~$ ros2 action send_goal /move_arm arm_broker_interfaces/action/
MoveArm "{joint_positions: [3.5, 0.0, 0.0, 0.0, 0.0, 0.0], client_id: 'prueba',
priority: 1}"
ros2 action send_goal /move_arm arm_broker_interfaces/action/MoveArm "{joint_pos
itions: [0.0, -2.3, -2.3, 1.2, 0.0, 0.0], client_id: 'prueba', priority: 1}"
ros2 action send_goal /move_arm arm_broker_interfaces/action/MoveArm "{joint_pos
itions: [1.5, 0.0, 0.0, 0.0, 0.0, 0.0], client_id: 'prueba', priority: 1}"
Waiting for an action server to become available...
Sending goal:
  joint_positions:
    - 3.5
    - 0.0
    - 0.0
    - 0.0
    - 0.0
    - 0.0
  client_id: prueba
  priority: 1
Goal was rejected.
Waiting for an action server to become available...
ces/action/MoveArm "{joint_positions:
11"

```

Figura 2.7(a). Primer rechazo en la CLI de ROS 2 (Goal was rejected): exceso en una articulación individual.

```

alumno01@rpi-02: ~
alumno01@rpi-02: ~ 80x24
Goal was rejected.
Waiting for an action server to become available...
Sending goal:
  joint_positions:
    - 0.0
    - -2.3
    - -2.3
    - 1.2
    - 0.0
    - 0.0
  client_id: prueba
  priority: 1
Goal was rejected.
Waiting for an action server to become available...
Sending goal:
  joint_positions:
    - 1.5
    - 0.0
    - 0.0
    - 0.0
    - 0.0
    - 0.0
ces/action/MoveArm "{joint_positions:
11"

```

Figura 2.7(b). Segundo rechazo: combinación articular que lleva el efector bajo el plano de trabajo ($z \leq 0$).

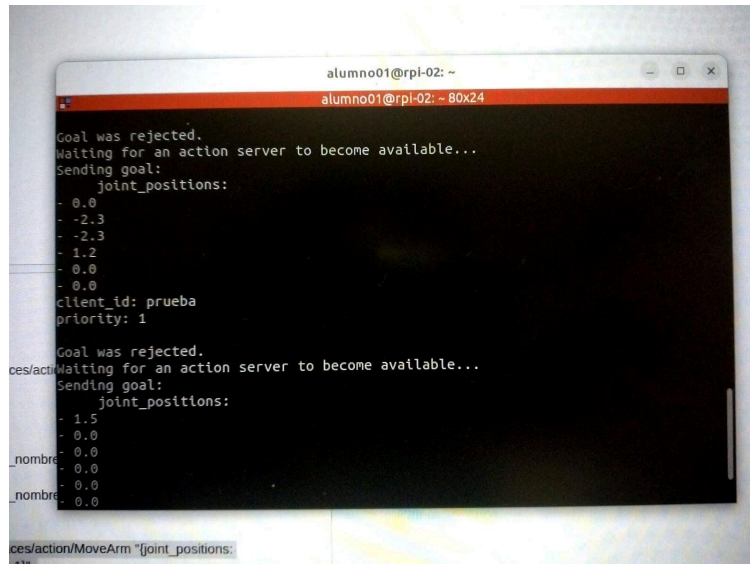


Figura 2.7(c). Tercer rechazo: paso articular excesivo respecto a la pose actual del brazo.

La CLI solo muestra «Goal was rejected». Para que cada rechazo exhiba su motivo, se adjuntan en [Github](#) as líneas RECHAZADO [cliente]: ... del log del broker correspondientes a los tres casos.

2.8 Política de cola: prioridad con envejecimiento y control de sobrecarga

Para evitar la inanición de los clientes de baja prioridad, la prioridad efectiva de cada meta i crece linealmente con su tiempo de espera. Con el convenio de MoveArm (mayor número = más urgente):

$$P_{ef,i}(t) = P_{base,i} + \Delta t_{espera,i} / \tau, \quad \Delta t_{espera,i} = t_{actual} - t_{arribo,i}, \quad \tau = 8.0 \text{ s}$$

El coeficiente de envejecimiento equivalente es $\alpha = 1/\tau = 0.125 \text{ s}^{-1}$. La política recorre las metas pendientes y selecciona la de mayor puntaje, usando la llegada más antigua como criterio de desempate; la política FIFO selecciona simplemente la de menor $t_{llegada}$. Como el término t_{actual}/τ es común a todas las metas, el orden relativo es invariante en el tiempo y podría indexarse con la clave estática $P_{base,i} - t_{arribo,i}/\tau$ en un min-heap con extracción $O(\log n)$. La implementación entregada realiza el recorrido lineal $O(n)$; con $cola_max = 20$ su costo es despreciable frente a los $\approx 3 \text{ s}$ de cada movimiento.

Una meta de prioridad 1 supera a una recién llegada de prioridad 5 tras $\Delta t^* = \tau \cdot (P_{max} - P_{min}) = 8.0 \cdot (5 - 1) = 32 \text{ s}$. Esta cantidad es la cota de referencia para el índice de inanición (sección 3.6).

Para acotar la memoria en el equipo embebido, el broker rechaza toda meta nueva cuando la cola alcanza $cola_max = 20$. La tasa de rechazo se audita con los contadores publicados:

$$R_{rate} = total_rejected / (total_accepted + total_rejected) \times 100 \%$$

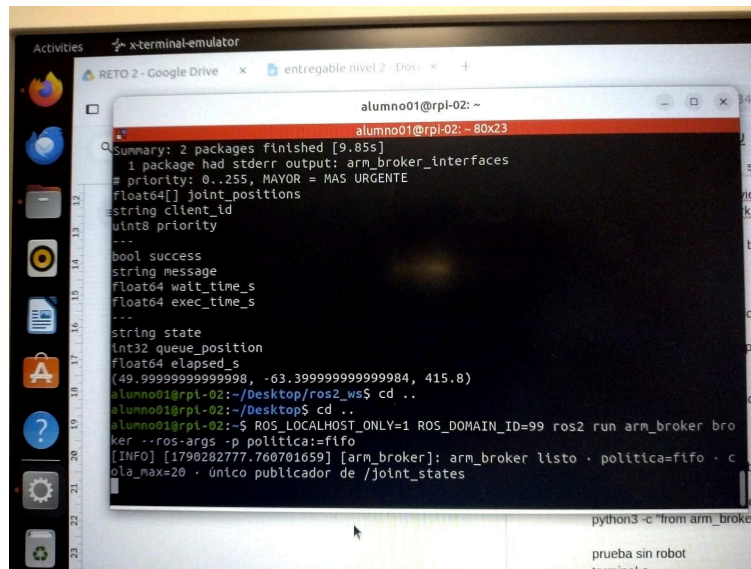
Tabla 2.9. Parámetros de la política y del broker.

Parámetro	Valor
Política (parámetro politica)	fifo o prioridad
Capacidad de cola (cola_max)	20
Constante de envejecimiento (τ , tau_envejecimiento_s)	8.0 s
Prioridades del ensayo (mayor = más urgente)	{1, 2, 3, 5} — ana P1, dario P2, carla P3, beto P5
Frecuencia de /arm/queue_state	5.0 Hz ($\Delta t \approx 0.2 \text{ s}$, desviación estándar 1.6 a 3.5 ms)
Tiempo de ejecución por meta	$\approx 3.02 \text{ a } 3.06 \text{ s}$ (exec_time_s medido)

2.9 Verificación funcional del broker

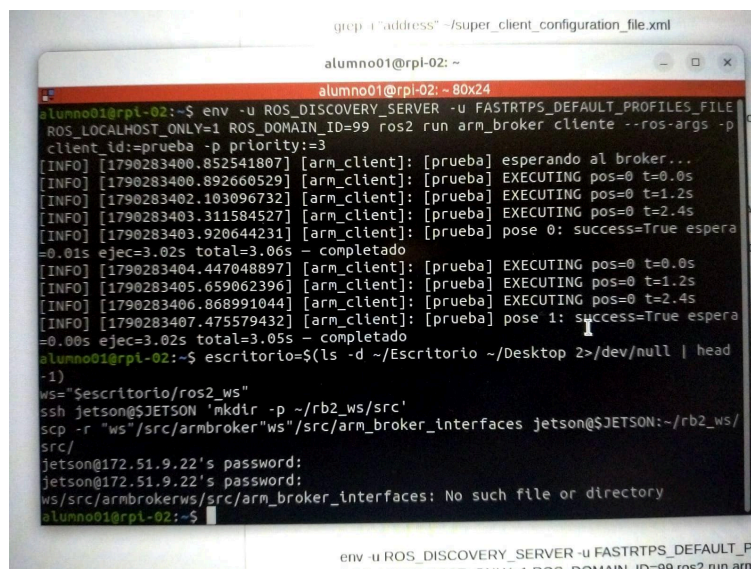
El broker se lanzó con la política FIFO y registró en consola que estaba listo, con capacidad de cola 20 y como único publicador de /joint_states (Figura 2.8). Un cliente de prueba (client_id «prueba», prioridad 3) recorrió el ciclo de vida de la acción: EXECUTING en $t = 0.0, 1.2$ y 2.4 s y finalización con success=True en ≈ 3.06 s por pose (Figura 2.9).

El código parchado se transfirió por SCP al workspace rb2_ws de la Jetson. Un primer intento falló por un error tipográfico en las rutas y se corrigió de inmediato. En el envío manual por CLI se detectó que el campo correcto del goal es joint_positions y no q_deseadas, que generaba un AttributeError (Figura 2.10).



```
Summary: 2 packages finished [9.85s]
1 package had stderr output: arm_broker_interfaces
# priority: 0..255, MAYOR = MAS URGENTE
Float64[] joint_positions
string client_id
uint8 priority
----
bool success
string message
float64 wait_time_s
float64 exec_time_s
----
string state
int32 queue_position
float64 elapsed_s
(49.99999999999998, -63.399999999999984, 415.8)
alumno01@rpi-02:~/Desktop/ros2_ws$ cd ..
alumno01@rpi-02:~/Desktop$ cd ..
alumno01@rpi-02:~$ ROS_LOCALHOST_ONLY=1 ROS_DOMAIN_ID=99 ros2 run arm_broker broker --ros-args -p politica:=fifo
[INFO] [1790282777.760701659] [arm_broker]: arm_broker listo - politica:=fifo - cola_max=20 - unico publicador de /joint_states
```

Figura 2.8. Arranque del nodo arm_broker en la Raspberry Pi con política FIFO, cola_max = 20 y aislamiento de red por variables de entorno.



```
grep -i "address" ~/super_client_configuration_file.xml
alumno01@rpi-02: ~
alumno01@rpi-02: ~80x24
alumno01@rpi-02:~$ env -u ROS_DISCOVERY_SERVER -u FASTRTPS_DEFAULT_PROFILES_FILE
ROS_LOCALHOST_ONLY=1 ROS_DOMAIN_ID=99 ros2 run arm_broker cliente --ros-args -p
client_id:=prueba -p priority:=3
[INFO] [1790283400.852541807] [arm_client]: [prueba] esperando al broker...
[INFO] [1790283400.892660529] [arm_client]: [prueba] EXECUTING pos=0 t=0.0s
[INFO] [1790283402.103096732] [arm_client]: [prueba] EXECUTING pos=0 t=1.2s
[INFO] [1790283403.311584527] [arm_client]: [prueba] EXECUTING pos=0 t=2.4s
[INFO] [1790283403.920644231] [arm_client]: [prueba] pose 0: success=True espera
=0.01s ejec=3.02s total=3.06s - completado
[INFO] [1790283404.447048897] [arm_client]: [prueba] EXECUTING pos=0 t=0.0s
[INFO] [1790283405.659062396] [arm_client]: [prueba] EXECUTING pos=0 t=1.2s
[INFO] [1790283406.868991044] [arm_client]: [prueba] EXECUTING pos=0 t=2.4s
[INFO] [1790283407.475579432] [arm_client]: [prueba] pose 1: success=True espera
=0.00s ejec=3.02s total=3.05s - completado
alumno01@rpi-02:~$ escritorio=$(ls -ld ~/Escritorio ~/Desktop 2>/dev/null | head
-1)
ws="Escritorio/ros2_ws"
ssh jetson@JETSON 'mkdir -p ~/rb2_ws/src'
scp -r "ws"/src/armbroker_ws/src/arm_broker_interfaces jetson@JETSON:~/rb2_ws/
src/
jetson@172.51.9.22's password:
jetson@172.51.9.22's password:
ws/src/armbroker_ws/src/arm_broker_interfaces: No such file or directory
alumno01@rpi-02:~$
```

Figura 2.9. Ejecución del cliente de acción (pose 0 y pose 1 con success=True) y transferencia del código por SCP al espacio ~/rb2_ws/src/ de la Jetson.

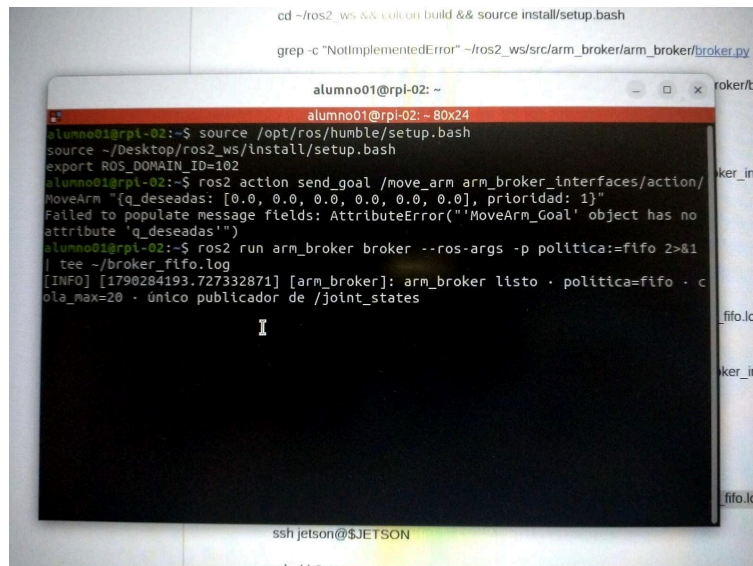


Figura 2.10(a). Auditoría de integridad del código parchado con grep y transferencia limpia por SCP hacia la Jetson.

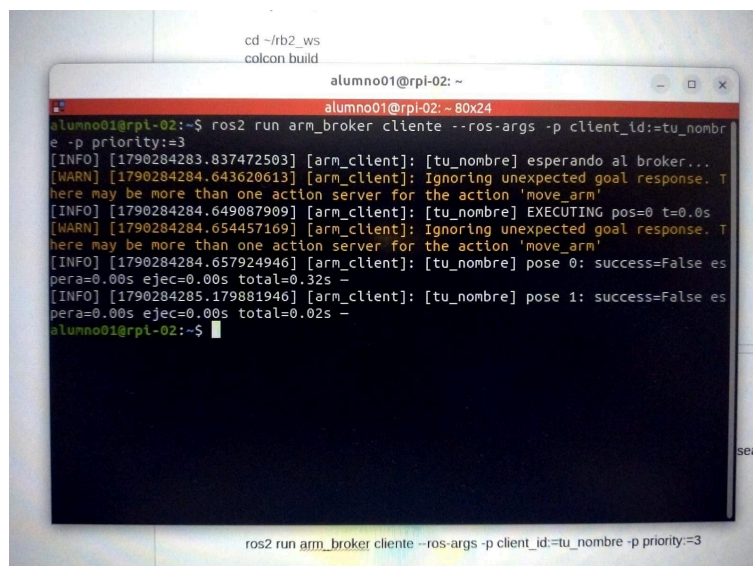


Figura 2.10(b). Depuración de la llamada CLI a la acción MoveArm: identificación del nombre correcto del campo joint_positions.

2.10 Pruebas de robustez

Tabla 2.10. Escenarios de falla provocados y resultado.

Escenario	Detección	Resultado	Evidencia
Dos instancias de arm_broker en el mismo dominio	Advertencia de más de un servidor para move_arm; Publisher count: 2 en /joint_states	Metas abortadas con success=False; tras reiniciar el demonio de ROS 2 el conteo bajó a Publisher count: 1	Figura 2.11
Cancelación global en caliente	Llamada al servicio /move_arm/_action/cancel_goal con solicitud vacía	return_code=0 y clientes finalizados con código de salida 1 controlado	Figura 2.12
Cierre de instancias previas	Señal SIGINT sobre arm_broker antes de cambiar de política	Liberación de /joint_states y /move_arm sin nodos huérfanos	Sección 3.2

La coexistencia de dos publicadores en /joint_states es exactamente la violación que el requisito de publicador único busca impedir. Su detección con el conteo de publicadores y su corrección (Figura 2.11)

forman parte de la evidencia de exclusión. En la cancelación (Figura 2.12), la respuesta del servicio confirma la recepción de la orden y la salida controlada de los clientes; el estado terminal de cada meta (CANCELED o ABORTED) se acredita con el log del broker.

```

alumno01@rpi-02: ~
alumno01@rpi-02: ~ 80x24
alumno01@rpi-02:~$ ros2 topic info /joint_states -v | grep -E "Publisher count|Node name"
Publisher count: 2
Node name: arm_broker
Node namespace: /
Node name: arm_broker
Node namespace: /
Node name: rosbag2_recorder
Node namespace: /
Node name: control_sync_plan
Node namespace: /
Node name: control_sync_plan
Node namespace: /
alumno01@rpi-02:~$

```

Figura 2.11(a). Inspección de /joint_states revelando dos publicadores activos (Publisher count: 2) por duplicidad de servidores.

```

alumno01@rpi-02: ~
alumno01@rpi-02: ~ 80x24
alumno01@rpi-02:~$ ros2 daemon stop; ros2 daemon start
sleep 20
ros2 topic info /joint_states -v | grep -E "Publisher count|Node name"
The daemon has been stopped
The daemon has been started
Publisher count: 1
Node name: arm_broker
Node namespace: /
Node name: rosbag2_recorder
Node namespace: /
Node name: control_sync_plan
Node namespace: /
Node name: control_sync_plan
Node namespace: /
alumno01@rpi-02:~$

```

Figura 2.11(b). Reinicio del demonio de ROS 2 y corrección del conteo a un único publicador legítimo (Publisher count: 1).

los CSV generados por exportar_csv.py a partir del bag (queue_state.csv y, opcionalmente, joint_states.csv) y se invoca con la carpeta de salida:

```
python3 verificar_exclusion.py prioridad_4clientes_csv/
```

El fundamento es que cada mensaje de /arm/queue_state publica un único executing_goal_id. Si el brazo atendiera dos pedidos a la vez, los identificadores se verían alternar (A, B, A) o un pedido aparecería ejecutándose y esperando en el mismo mensaje. Recorriendo los mensajes en orden temporal, el script reconstruye para cada meta g el intervalo de ejecución $I_g = [s_g, e_g]$ (primer y último mensaje en que figura como ejecutándose) y cuenta como violación cada una de las siguientes situaciones:

1. **V₁**: un goal_id que figura a la vez como ejecutándose y en la lista de encolados del mismo mensaje.
2. **V₂**: un goal_id que reaparece como ejecutándose después de que otro tomó el brazo (alternancia A, B, A).
3. **V₃**: dos intervalos consecutivos, ordenados por inicio, que se solapan: $s_b < e_a$.

$$V = V_1 + V_2 + V_3 \quad (\text{exclusión mutua certificada} \Leftrightarrow V = 0)$$

Además informa el número de mensajes, los goals distintos, los clientes que ejecutaron, la fracción de mensajes con el brazo ocupado y el mayor salto articular entre muestras consecutivas de /joint_states; este último no es una violación, pero permite mostrar que ningún otro publicador introdujo discontinuidades en la trayectoria. Si $V > 0$, el script termina con código de salida 1; si $V = 0$, imprime «VIOLACIONES DE EXCLUSION MUTUA: 0 -> CERO (cumple)».

Alcance del instrumento. Dado que cada mensaje contiene un solo executing_goal_id, la herramienta detecta alternancias, reapariciones y solapamientos de intervalos, pero no puede representar dos ejecuciones simultáneas dentro de un mismo mensaje, y la resolución temporal es la del muestreo a 5 Hz (0.2 s). Por ello el resultado se complementa con la garantía estructural del worker único (sección 2.6) y con el conteo de publicadores de /joint_states (Figura 2.11b).

Tabla 2.11. Criterios de certificación de exclusión mutua.

Criterio	Fuente	Valor exigido	Resultado
V ₁ : goal ejecutándose y encolado a la vez	queue_state.csv	0	Adjuntar salida de verificar_exclusion.py
V ₂ : reaparición tras ceder el brazo	queue_state.csv	0	Adjuntar salida de verificar_exclusion.py
V ₃ : solapamiento de intervalos de ejecución	queue_state.csv	0	Adjuntar salida de verificar_exclusion.py
Salto articular máximo entre muestras	joint_states.csv	Sin discontinuidades	Adjuntar salida de verificar_exclusion.py
Publicadores en /joint_states	ros2 topic info	1	1 (Figura 2.11b)

Figura 2.13 (por adjuntar). Captura de la salida de verificar_exclusion.py sobre prioridad_4clientes_csv/ y fifo_4clientes_csv/, con la línea final «VIOLACIONES DE EXCLUSION MUTUA: 0 -> CERO (cumple)».

Ítem 3 — Medición bajo contención

El Ítem 3 compara dos políticas de cola, FIFO y prioridad con envejecimiento ($\tau = 8.0$ s), bajo la misma carga concurrente de cuatro clientes. Se graba con ros2 bag, se exporta a CSV y se consolidan las métricas de espera, inanición, equidad y rechazos. Antes de medir en hardware se obtuvo una línea base teórica con simulación, de modo que el resultado experimental se contrasta con una hipótesis y no se ajusta a posteriori.

3.1 Objetivo y diseño experimental

Los cuatro integrantes lanzaron en paralelo un cliente propio contra el mismo servidor /move_arm, de modo que las condiciones fueran idénticas para ambas políticas. Cada cliente opera en lazo cerrado: envía una meta, espera el resultado y, tras una breve pausa, envía la siguiente.

Tabla 3.1. Diseño experimental.

Factor	Valor
Políticas comparadas	FIFO; prioridad con envejecimiento ($\tau = 8.0$ s)
Clientes y prioridades (mayor = más urgente)	ana P1, dario P2, carla P3, beto P5
Carga por cliente	2 poses $\times$ 3 repeticiones = 6 metas; 4 clientes concurrentes = 24 metas (más una meta de reseteo previa)
Capacidad de cola (cola_max)	20
Registro	ros2 bag de /arm/queue_state y /joint_states
Conjuntos de datos	fifo_4clientes y prioridad_4clientes
Muestras de prioridad_4clientes	1,319 mensajes de /arm/queue_state (≈ 264 s de captura) y 240 muestras de /joint_states
Frecuencia de /arm/queue_state	5.0 Hz ($\Delta t \approx 0.2$ s; desviación estándar 1.6 a 3.5 ms)

3.2 Protocolo de ejecución

Cada ensayo siguió un protocolo de cuatro fases (Figura 3.1):

1. **Reinicio limpio.** Se envió SIGINT a cualquier instancia previa de arm_broker para liberar /joint_states y /move_arm, evitando nodos huérfanos que distorsionen la medición, y se lanzó el servidor en la Jetson con la política correspondiente.
2. **Homologación de pose.** Una meta con client_id «reset» devolvió las articulaciones a [0, 0, 0, 0, 0, 0] rad con estado SUCCEEDED (wait_time_s ≈ 0.0009 s y exec_time_s ≈ 3.029 s).
3. **Persistencia.** Se inició la grabación del bag sobre /arm/queue_state y /joint_states.
4. **Carga concurrente.** Se lanzaron en segundo plano los cuatro clientes (ana, beto, carla, dario) con una espera común y un log independiente por cliente.

Tras la ejecución se verificó la completitud: cada cliente de prioridad_4clientes registró exactamente 6 metas con success=True, sin pérdidas (24 de 24). El estado final de /arm/queue_state reportó total_accepted = 25 (24 metas más el reseteo), total_rejected = 0 y total_completed = 25.

```
alumno01@rpi-02:~$ ros2 daemon stop; ros2 daemon start
sleep 15
ros2 topic info /joint_states -v | grep -E "Publisher count|Node name"
ros2 action send_goal /move_arm arm_broker_interfaces/action/MoveArm "{joint_pos
titions: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0], client_id: 'reset', priority: 1}"
The daemon has been stopped
The daemon has been started
Publisher count: 1
Node name: arm_broker
Node namespace: /
Node name: rosbag2_recorder
Node namespace: /
Node name: control_sync_plan
Node namespace: /
Node name: control_sync_plan
Node namespace: /
Waiting for an action server to become available...
Sending goal:
  joint_positions:
  - 0.0
  - 0.0
  - 0.0
  - 0.0
  - 0.0
  - 0.0
  client_id: reset
  priority: 1
Goal accepted with ID: 5296097e68484c1f99193f0622f13283
Result:
  success: true
  message: completado
  wait_time_s: 0.0008373260498046875
  exec_time_s: 3.029367208480835
Goal finished with status: SUCCEEDED
alumno01@rpi-02:~$
```

Figura 3.1(a). Flujo del ensayo de prioridad: reinicio del demonio, verificación de un único publicador y envío de la meta de reseteo (success: true, SUCCEEDED).

```
The daemon has been stopped
The daemon has been started
Publisher count: 1
Node name: arm_broker
Node namespace: /
Node name: rosbag2_recorder
Node namespace: /
Node name: control_sync_plan
Node namespace: /
Node name: control_sync_plan
Node namespace: /
Waiting for an action server to become available...
Sending goal:
  joint_positions:
  - 0.0
  - 0.0
  - 0.0
  - 0.0
  - 0.0
  - 0.0
  client_id: reset
  priority: 1
Goal accepted with ID: 5296097e68484c1f99193f0622f13283
Result:
  success: true
  message: completado
  wait_time_s: 0.0008373260498046875
  exec_time_s: 3.029367208480835
Goal finished with status: SUCCEEDED
alumno01@rpi-02:~$ mkdir -p ~/evidencias && cd ~/evidencias
ros2 bag record -o prioridad_4clientes /arm/queue_state /joint_states
[INFO] [1790289065.961456319] [rosbag2_recorder]: Press SPACE for pausing/resumi
ng
[INFO] [1790289065.967763782] [rosbag2_storage]: Opened database 'prioridad_4cli
entes/prioridad_4clientes_0.db3' for READ_WRITE.
[INFO] [1790289065.970301448] [rosbag2_recorder]: Listening for topics...
[INFO] [1790289065.970359436] [rosbag2_recorder]: Event publisher thread: Starti
ng
[INFO] [1790289065.970882504] [rosbag2_recorder]: Recording...
[INFO] [1790289066.488229800] [rosbag2_recorder]: Subscribed to topic '/arm/que
ue_state'
[INFO] [1790289066.494547041] [rosbag2_recorder]: Subscribed to topic '/joint_s
tates'
[INFO] [1790289066.494689596] [rosbag2_recorder]: All requested topics are subs
cribed. Stopping discovery...
```

Figura 3.1(b). Resultado del reseteo, inicio de la grabación con ros2 bag record sobre /arm/queue_state y /joint_states.

3.3 Justificación metodológica de la prueba multi-terminal

Los cuatro clientes se lanzaron desde cuatro terminales independientes de la misma Raspberry Pi (alumno01@rpi-02), mientras el broker se ejecutó en la Jetson. Esta configuración produce en el servidor

la misma contención que cuatro computadoras separadas y no introduce sesgo en las métricas, por las razones que se desarrollan a continuación.

Fundamento formal

El broker es un sistema determinista respecto de la secuencia de arribos. Para cada goal, solo observa la terna (client_id, priority, t_arribo), donde t_arribo se estampa con **su propio reloj**. Por tanto, la espera de cada meta es función únicamente de esa secuencia, del tiempo de servicio S y de la política: $W = f(\text{arribos}, S, \text{política})$. La topología física de los clientes solo puede influir a través del desfase de arribo δ (inicio de los procesos y latencia de transporte). Dicho desfase es del orden de decenas de milisegundos, y su efecto sobre el orden de servicio es acotado:

$$\Delta P_{ef} = \delta / \tau \approx 0.05 \text{ s} / 8.0 \text{ s} \approx 0.006 \ll \Delta P_{min} = 1$$

Es decir, un retraso de 50 ms modifica la prioridad efectiva en 0.006 unidades, más de dos órdenes de magnitud por debajo de la separación mínima entre las prioridades de los clientes ({1, 2, 3, 5}). El orden por prioridad al momento del arribo es, entonces, invariante frente a la topología; y como $\delta/S \approx 0.05 / 3.02 < 2 \%$, tampoco altera la secuencia de despacho entre metas de igual prioridad en más de una posición.

Pilares de validez

1. **Aislamiento de procesos POSIX.** Cada terminal ejecuta su propia instancia del cliente como un proceso con PID propio, espacio de memoria virtual independiente y pila propia; no existe memoria compartida entre clientes. Cada proceso rclpy tiene además su propio intérprete de Python y su propio GIL. Si los clientes fueran hilos de un mismo proceso, el GIL serializaría su ejecución y distorsionaría los arribos; como procesos separados, el planificador del kernel los ejecuta de forma realmente concurrente.
2. **Desacoplamiento en la capa DDS/ROS 2.** Cada cliente instancia un nodo único (por ejemplo /arm_client_ana, /arm_client_beto, /arm_client_carla y /arm_client_dario) con su participante DDS de GUID propio, su cliente de acción rclpy y sockets UDP dedicados, registrados en el Discovery Server. La contención no ocurre en la terminal, sino en la cola del servidor de acciones /arm_broker, que procesa las peticiones exactamente igual que si provinieran de cuatro placas distintas conectadas a la red.
3. **Inducción de condiciones de carrera y saturación.** El lanzamiento en segundo plano con espera común produce una ráfaga casi simultánea hacia /move_arm, el caso de máxima contención: la demanda de cuatro clientes supera la capacidad de un servidor que atiende una meta cada $S \approx 3.02$ s. Esto obliga a la lógica de admisión, filtrado por FK, encolado y envejecimiento a operar con cola no vacía y exige que la exclusión mutua se cumpla bajo concurrencia real. La carga total ($\approx 24 \times 3.02$ s ≈ 72.5 s) supera la cota de envejecimiento de 32 s, de modo que el efecto de $\tau = 8.0$ s se manifiesta dentro de un mismo ensayo. Al ser un lazo cerrado, a lo sumo cuatro metas esperan a la vez, por lo que se satura la capacidad de servicio y no la capacidad de la cola (cola_max = 20), coherente con total_rejected = 0.
4. **Trazabilidad y resguardo de evidencias.** Toda la interacción se registró en tiempo real con ros2 bag y en un log por cliente, lo que permite auditar que las cuatro identidades operaron bajo sus prioridades asignadas (P1, P2, P3 y P5).

Tabla 3.2. Equivalencia entre clientes en terminales independientes y clientes en máquinas separadas.

Aspecto	Terminales independientes en rpi-02	Computadoras separadas	Efecto sobre la medición
Identidad de proceso	PID y memoria virtual propios	Ídem	Ninguno

Aspecto	Terminales independientes en rpi-02	Computadoras separadas	Efecto sobre la medición
Nodo y participante DDS	Nodo y GUID propios	Ídem	Ninguno
Identificación en el broker	client_id y priority del goal	Ídem	Ninguno
Reloj de espera	Reloj del broker	Reloj del broker	Ninguno
Transporte	Interfaz de red compartida	Interfaz propia	$\epsilon_{red} \ll S$ ($< 1\%$ de S)
Sincronía de arribo	Desfase de decenas de ms	Desfase variable	$\Delta P_{ef} \approx 0.006 \ll 1$

Amenazas a la validez y mitigación. Los cuatro clientes comparten CPU e interfaz de red de rpi-02. Como pasan casi todo el tiempo bloqueados esperando el resultado y el broker corre en la Jetson, su carga de CPU es marginal. Aun así, se declara la topología de cada ensayo y se recomienda repetir cada política al menos tres veces, reportando media y desviación estándar.

3.4 Instrumentación: línea base teórica y pipeline de datos

Simulación previa con predecir_p95.py

El script simula, sin ROS ni robot, la espera por política antes de medir en hardware. Modela cuatro clientes en lazo cerrado (envían una meta, esperan el resultado, descansan τ segundos y envían la siguiente), un brazo que atiende de a una meta con tiempo de servicio fijo y no modela red, rechazos ni cancelaciones. Carga el módulo políticas.py real, por lo que la predicción cambia si cambian τ o la lógica de selección. Entrega, por prioridad, la espera media, el P95 y el máximo, además del índice de inanición y la equidad de Jain. Se ejecutó con los parámetros del ensayo:

```
python3 predecir_p95.py --src ~/ros2_ws/src/arm_broker --poses 6 --servicio 3.02 --pausa 0.5 --tau 8.0
```

La línea base resultante (Tabla 3.3) anticipa dos comportamientos: bajo FIFO la espera es independiente de la prioridad (≈ 7 a 9 s para todos los clientes) y bajo prioridad con envejecimiento la espera se ordena por prioridad, con una inanición de P1 de ≈ 27 s, por debajo de la cota de 32 s.

Tabla 3.3. Línea base teórica (predecir_p95.py): 4 clientes $\times$ 6 metas, servicio 3.02 s, pausa 0.5 s, $\tau = 8.0$ s.

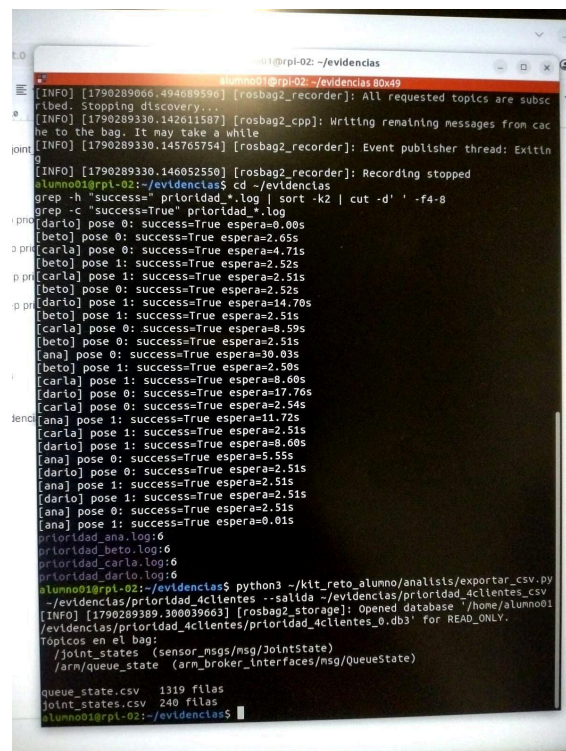
Métrica	Cliente / prioridad	FIFO (predicho)	Prioridad + aging (predicho)
Espera media (s)	P1 (ana)	7.13	8.64
Espera media (s)	P2 (dario)	8.64	8.14
Espera media (s)	P3 (carla)	8.14	4.62
Espera media (s)	P5 (beto)	7.64	2.10
P95 de espera (s)	P1 (ana)	8.56	27.18
P95 de espera (s)	P2 (dario)	9.06	17.62
P95 de espera (s)	P3 (carla)	8.56	8.56
P95 de espera (s)	P5 (beto)	8.56	2.52
Global	Espera media / P95 (s)	7.89 / 8.56	5.88 / 17.62
Índice de inanición (s)	Espera máxima de P1	8.56	27.18
Índice de Jain	Goals por cliente	1.000	1.000

Con $n = 6$ observaciones por cliente, el estimador de percentil de los scripts (índice redondeado de $0.95 \cdot (n - 1)$) coincide con el máximo; por eso el P95 por cliente equivale a la espera máxima observada.

Pipeline de extracción con exportar_csv.py

Los bags de rosbag2 se convierten a CSV con exportar_csv.py, que abre el bag con rosbag2_py (almacenamiento sqlite3, archivos .db3), deserializa cada mensaje con su tipo registrado y genera una serie temporal por tópico en la carpeta indicada con la opción de salida. Para leer bags en formato .mcap bastaría cambiar el identificador de almacenamiento del lector. El archivo queue_state.csv contiene, por mensaje, el instante en nanosegundos, la meta en ejecución (cliente, identificador y tiempo transcurrido), la longitud de la cola, las listas de identificadores, clientes, prioridades y esperas de los encolados, y los contadores total_accepted, total_rejected y total_completed; joint_states.csv contiene las seis posiciones articulares j1 a j6.

Para prioridad_4clientes se extrajeron 1,319 filas de queue_state.csv y 240 de joint_states.csv. La estabilidad temporal del tópico se comprobó midiendo su frecuencia con ros2 topic hz sobre /arm/queue_state: 5.0 Hz con desviación estándar entre 1.6 y 3.5 ms (Figura 3.2).



```
alumno01@rpi-02: ~/evidencias
[INFO] [1790289306.404895596] [rosbag2_recorder]: All requested topics are subscribed. Stopping discovery...
[INFO] [1790289330.142611587] [rosbag2_cpp]: Writing remaining messages from cache to the bag. It may take a while
[INFO] [1790289330.145765754] [rosbag2_recorder]: Event publisher thread: Exiting
[INFO] [1790289330.146052550] [rosbag2_recorder]: Recording stopped
alumno01@rpi-02: ~/evidencias$ cd ~/evidencias
grep -h "success=" prioridad*.log | sort -k2 | cut -d' ' -f4-8
grep -c "success=True" prioridad*.log
prio [darto] pose 0: success=True espera=0.00s
prio [beto] pose 0: success=True espera=2.65s
prio [carla] pose 0: success=True espera=4.71s
prio [beto] pose 1: success=True espera=2.52s
prio [carla] pose 1: success=True espera=2.51s
prio [beto] pose 0: success=True espera=2.52s
prio [darto] pose 1: success=True espera=14.70s
prio [beto] pose 1: success=True espera=2.51s
prio [carla] pose 0: success=True espera=8.59s
prio [beto] pose 0: success=True espera=2.51s
prio [ana] pose 0: success=True espera=30.03s
prio [beto] pose 1: success=True espera=2.50s
prio [carla] pose 1: success=True espera=8.60s
prio [darto] pose 0: success=True espera=17.70s
prio [carla] pose 0: success=True espera=2.54s
prio [ana] pose 1: success=True espera=11.72s
prio [carla] pose 1: success=True espera=2.51s
prio [darto] pose 1: success=True espera=8.60s
prio [ana] pose 0: success=True espera=5.55s
prio [darto] pose 0: success=True espera=2.51s
prio [ana] pose 1: success=True espera=2.51s
prio [darto] pose 1: success=True espera=2.51s
prio [ana] pose 0: success=True espera=0.01s
prio prioridad_ana.log:6
prio prioridad_beto.log:6
prio prioridad_carla.log:6
prio prioridad_darto.log:6
alumno01@rpi-02: ~/evidencias$ python3 ~/kit_reto_alumno/analysis/exportar_csv.py
~/evidencias/prioridad_4clientes --salida ~/evidencias/prioridad_4clientes_csv
[INFO] [1790289389.300039663] [rosbag2_storage]: Opened database ~/home/alumno01/evidencias/prioridad_4clientes/prioridad_4clientes_0.db3 for READ_ONLY.
Tópicos en el bag:
/joint_states (sensor_msgs/msg/JointState)
/arm/queue_state (arm_broker_interfaces/msg/QueueState)
queue_state.csv 1319 filas
joint_states.csv 240 filas
alumno01@rpi-02: ~/evidencias$
```

Figura 3.2(a). Postprocesamiento del ensayo de prioridad: verificación de los logs por cliente (6 metas con success=True por cliente) y esperas registradas.

Figura 3.2(b). Conversión del bag a CSV con `exportar_csv.py` (`queue_state.csv`: 1,319 filas; `joint_states.csv`: 240 filas) y comprobación de la tasa de refresco de 5 Hz.

Consolidación de métricas con `metricas.py`

El script `metricas.py` lee un `queue_state.csv` por política y calcula, para cada una, la espera media y el P95 por prioridad, el índice de inanición y la equidad de Jain, además de una figura comparativa. La espera de cada meta se toma como el mayor valor observado de `queued_wait_s` mientras la meta permanece en la cola; por ello la resolución es la del muestreo a 5 Hz (0.2 s) y una meta atendida de inmediato puede no aparecer en la cola. Para obtener series completas, las esperas de la Tabla 3.4 provienen del campo `wait_time_s` que cada cliente registra en su log, que cubre las 24 metas.

Definición de métricas

- **Espera media por prioridad, $W_{\square}$, y percentil P95** de las esperas de cada prioridad (percentil por rango más cercano).
- **Índice de inanición:** espera máxima de la prioridad más baja (P1).
- **Tasa de rechazos:** R_{rate} , definida en la sección 2.8.
- **Índice de equidad de Jain:** $J = (\sum x_i)^2 / (n \cdot \sum x_i^2)$, con x_i los goals atendidos por cliente y $n = 4$. Cumple $1/n \leq J \leq 1$; $J = 1$ indica un reparto perfectamente equitativo. Se reporta también calculado sobre las esperas medias por cliente para discriminar el efecto de cada política.

3.5 Resultados medidos

La Tabla 3.4 consolida las métricas del ensayo real de prioridad con envejecimiento y las contrasta con la línea base teórica de ambas políticas. Las celdas resaltadas del ensayo FIFO corresponden a mediciones que deben completarse con los logs y el bag de `fifo_4clientes`.

Tabla 3.4. Resultados consolidados bajo contención: línea base teórica frente a ensayo real.

Métrica	Cliente / prioridad	FIFO predicho	FIFO medido	Prioridad + aging predicho	Prioridad + aging medido
Espera media (s)	P1 (ana)	7.13	por medir	8.64	8.72
Espera media (s)	P2 (dario)	8.64	por medir	8.14	7.68
Espera media (s)	P3 (carla)	8.14	por medir	4.62	4.91
Espera media (s)	P5 (beto)	7.64	por medir	2.10	2.54
P95 de espera (s)	P1 (ana)	8.56	por medir	27.18	30.03
P95 de espera (s)	P2 (dario)	9.06	por medir	17.62	17.76
P95 de espera (s)	P3 (carla)	8.56	por medir	8.56	8.60
P95 de espera (s)	P5 (beto)	8.56	por medir	2.52	2.65
Global	Espera media (s)	7.89	por medir	5.88	5.96
Global	P95 de espera (s)	8.56	por medir	17.62	17.76
Índice de inanición (s)	Espera máxima de P1	8.56	por medir	27.18	30.03
Metas completadas / enviadas	Total	24 / 24	por medir	24 / 24	24 / 24
Tasa de rechazos R_rate (%)	Total	—	por medir	—	0.0 (0 de 25)
Índice de Jain	Goals por cliente	1.000	por medir	1.000	1.000 (6/6/6/6)
Índice de Jain	Esperas medias por cliente	0.995	por medir	0.828	0.859
Violaciones de exclusión mutua	Global	—	por medir	—	ver Tabla 2.11

El detalle de las esperas que sustenta las columnas medidas de prioridad se presenta en la Tabla 3.5; proviene de los logs de los cuatro clientes mostrados en la Figura 3.2(a).

Tabla 3.5. Esperas registradas por meta en el ensayo de prioridad (wait_time_s, en segundos).

Cliente	Prioridad	Esperas de las 6 metas (de mayor a menor)	Media	P95 (= máx.)
ana	P1	30.03 · 11.72 · 5.55 · 2.51 · 2.51 · 0.01	8.72	30.03
dario	P2	17.76 · 14.70 · 8.60 · 2.51 · 2.51 · 0.00	7.68	17.76
carla	P3	8.60 · 8.59 · 4.71 · 2.54 · 2.51 · 2.51	4.91	8.60
beto	P5	2.65 · 2.52 · 2.52 · 2.51 · 2.51 · 2.50	2.54	2.65

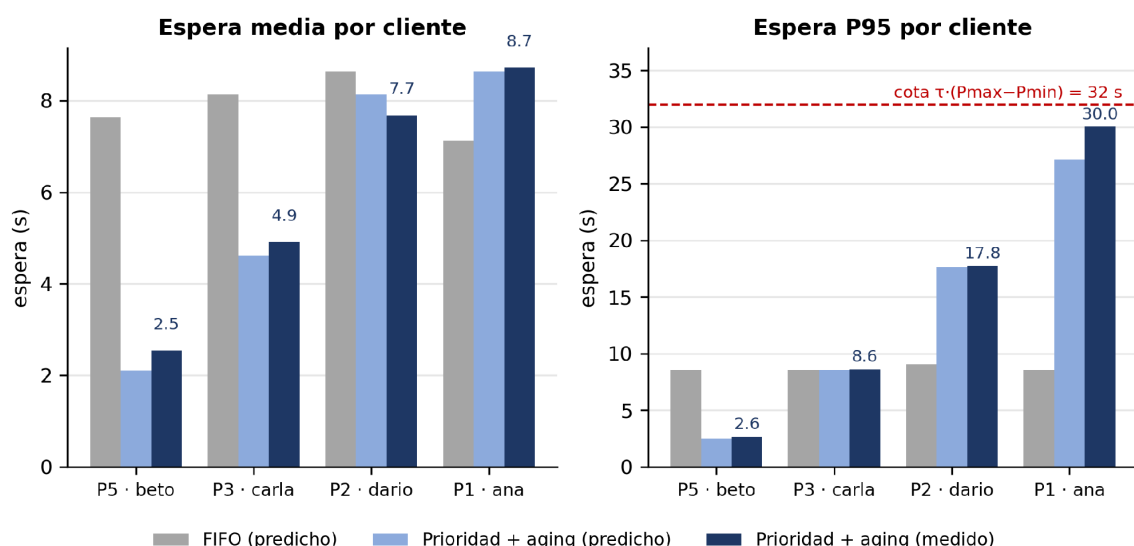


Figura 3.3. Espera media y P95 por cliente: línea base teórica (FIFO y prioridad con aging) frente al ensayo real de prioridad con aging.

3.6 Análisis de resultados

Los resultados se interpretan con criterios fijados antes de medir (sección 2.8):

- **El orden por prioridad se cumple.** Con envejecimiento, la espera media crece monótonamente al bajar la prioridad: 2.54 s (P5) < 4.91 s (P3) < 7.68 s (P2) < 8.72 s (P1), por lo que $W(P5) < W(P1)$, tal como predice el modelo.
- **No hay inanición.** La espera máxima de la prioridad más baja fue de 30.03 s, inferior a la cota de referencia $\tau \cdot (P_{max} - P_{min}) + S \approx 32$ s + 3 s ≈ 35 s. El envejecimiento cumple su función: ana, con la prioridad más baja, fue atendida en todas sus metas.
- **La predicción se sostiene.** Las esperas medias medidas difieren de la línea base en 0.08 a 0.46 s y los P95 en 0.04 a 0.14 s, salvo P1 (+2.85 s). Esa desviación es del orden de un tiempo de servicio ($S \approx 3.02$ s), es decir, una posición en la secuencia de despacho, y los desfases de arribo y la latencia de red, no modelados, bastan para producirla.
- **Compromiso frente a FIFO.** Frente a la línea base de FIFO, la prioridad con envejecimiento reduce la espera media global de 7.89 s a 5.96 s (−24 %), a costa de ensanchar la cola de la distribución: el P95 global pasa de 8.56 s a 17.76 s. La inequidad de esperas es deliberada: el Jain sobre esperas medias baja de 0.995 a 0.859.
- **Sobre la equidad.** Como todos los clientes completan sus 6 metas, el Jain sobre goals atendidos vale 1.000 en ambas políticas y no las discrimina; la diferencia aparece al calcularlo sobre los tiempos de espera.
- **Sin rechazos.** total_rejected = 0 sobre 25 metas aceptadas: con cuatro clientes en lazo cerrado la cola nunca se acerca a cola_max = 20.

La conclusión sobre la política a llevar a CapyTown se desarrolla en el cierre reflexivo.

Ítem 4 — La puerta a la cinemática inversa

Un único objetivo cartesiano (x, y, z) sobre el tablero de la mesa, resuelto con `send_coords()` de `pymycobot`: la cinemática inversa la resuelve el firmware, el equipo no escribe solver. Luego leen el vector articular q realmente ejecutado, calculan $FK(q)$ con su propio código y miden el error contra lo solicitado.

La cinemática directa que construyeron se convierte en el instrumento que audita una cinemática inversa ajena. Queda abierta la pregunta que inicia la semana 5: ¿por qué el brazo eligió esa solución y no la del codo contrario?

4.1 Procedimiento experimental

Para evaluar la cinemática inversa (IK) integrada en el firmware del JetCobot se utilizó la función `fk(q)` desarrollada y validada en el Ítem 1 como instrumento de auditoría.

El ensayo comienza con la selección del punto objetivo $P_{meta} = [218.50, -76.30, 285.70]$ mm, definido dentro del espacio de trabajo del robot respecto al marco de su base. Para no inducir reorientaciones innecesarias de la muñeca durante el movimiento, se lee previamente la orientación actual $[rx, ry, rz]$ con `get_coords()` y se concatena con la posición objetivo.

El vector de seis variables cartesianas $[x, y, z, rx, ry, rz]$ se envía al JetCobot con la función `send_coords()` de la librería `pymycobot`, a velocidad 15 y en modo de trayectoria lineal (`mode = 1`). El solver de cinemática inversa del firmware calcula y ejecuta de forma autónoma la configuración articular necesaria. Después del envío se espera 8 s para que los servomotores se detengan antes de leer los ángulos.

Finalmente se lee el vector de ángulos articulares realmente alcanzado, $q_{ejecutado}$, con `get_angles()`. Este vector q se usa como entrada de `fk(q)` en MATLAB para obtener la posición predicha P_{FK} y calcular las desviaciones, totales y por eje, respecto a la meta.

4.2 Medición en el JetCobot

A continuación se presenta el script en Python ejecutado en el JetCobot. El código lee con `get_coords()` la orientación actual $[rx, ry, rz]$ para conservarla, envía la meta con `send_coords()` a velocidad 15 en modo lineal (`mode = 1`), espera 8 s para la estabilización del brazo y captura con `get_angles()` el vector de ángulos q resultante:

```
import time

meta = [218.5, -76.3, 285.7]
objetivo = meta

x, y, z = objetivo

coords = mc.get_coords()
rx, ry, rz = coords[3], coords[4], coords[5]

mc.send_coords([x, y, z, rx, ry, rz], 15, 1)

time.sleep(8)

q = mc.get_angles()

print("meta:", objetivo)
print("angulos:", q)
```

```

meta = [218.5, -76.3, 285.7]

x, y, z = objetivo

coords = mc.get_coords()
rx, ry, rz = coords[3], coords[4], coords[5]

mc.send_coords([x, y, z, rx, ry, rz], 15, 1)

time.sleep(8)

q = mc.get_angles()

print("meta:", objetivo)
print("angulos:", q)

meta: [218.5, -76.3, 285.7]
angulos: [-5.09, -85.51, 50.44, 19.16, 5.8, -45.7]

```

Figura 4.1. Ejecución del comando cartesiano en Python y ángulos articulares q resultantes reportados por el JetCobot.

4.3 Resultados de la respuesta articular (IK del firmware)

A partir de la ejecución del script, el solver de cinemática inversa (IK) del firmware del JetCobot procesó el objetivo $P_meta = [218.50, -76.30, 285.70]$ mm y determinó la configuración articular requerida. La Tabla 4.1 presenta los ángulos $q = [q_1, q_2, q_3, q_4, q_5, q_6]$ leídos con `get_angles()` una vez estabilizado el brazo:

Tabla 4.1. Ángulos articulares q reportados por el JetCobot tras ejecutar la IK.

Articulación	Variable	Ángulo medido (°)
J1	q_1	-5.09
J2	q_2	-85.51
J3	q_3	50.44
J4	q_4	19.16
J5	q_5	5.80
J6	q_6	-45.70

4.4 Comprobación con la $fk(q)$ propia

Con el vector articular ejecutado por el JetCobot, $q = [-5.09, -85.51, 50.44, 19.16, 5.80, -45.70]^\circ$, se calculó en MATLAB la posición predicha por el modelo propio (FK) mediante la función `fk(q)` validada en el Ítem 1, basada en la tabla DH. A partir de ella se obtuvieron el error de posición euclidiano y las desviaciones por eje respecto a la meta $[218.50, -76.30, 285.70]$ mm:

```

clc;
clear;
close all;

q = [-5.09 -85.51 50.44 19.16 5.80 -45.70];

meta = [218.5 -76.3 285.7];

pos = fk(q);

error_x = abs(pos(1) - meta(1));
error_y = abs(pos(2) - meta(2));
error_z = abs(pos(3) - meta(3));
error_cartesiano = norm(pos - meta);

fprintf('q = [%2f %2f %2f %2f %2f %2f] grados\n', q);
fprintf('meta = [%2f %2f %2f] mm\n', meta);

```

```

fprintf('fk(q) = [%2f %2f %2f] mm\n', pos);
fprintf('error x = %2f mm\n', error_x);
fprintf('error y = %2f mm\n', error_y);
fprintf('error z = %2f mm\n', error_z);
fprintf('error cartesiano = %2f mm\n', error_cartesiano);

function pos = fk(q)
d1 = 134.75;
a2 = -110;
a3 = -96;
d4 = 63.4;
d5 = 75.05;
d6 = 50;

a1 = dh(q(1), d1, 0, 90);
a2_matriz = dh(q(2) - 90, 0, a2, 0);
a3_matriz = dh(q(3), 0, a3, 0);
a4 = dh(q(4) - 90, d4, 0, 90);
a5 = dh(q(5) + 90, d5, 0, -90);
a6 = dh(q(6), d6, 0, 0);

t06 = a1 * a2_matriz * a3_matriz * a4 * a5 * a6;
pos = t06(1:3, 4)';
end

function A = dh(theta, d, a, alpha)
A = [cosd(theta), -sind(theta)*cosd(alpha), ...
    sind(theta)*sind(alpha), a*cosd(theta);
    sind(theta), cosd(theta)*cosd(alpha), ...
    -cosd(theta)*sind(alpha), a*sind(theta);
    0, sind(alpha), cosd(alpha), d;
    0, 0, 0, 1];
end

```

The screenshot shows a MATLAB script being executed. The script defines a function `fk(q)` that calculates the position of a robotic arm based on joint angles `q` and link parameters. It also defines a function `dh(theta, d, a, alpha)` for the Denavit-Hartenberg transformation. The main script sets joint angles `q` and link parameters `meta`, then calls `fk(q)` to calculate the position `pos`. It then calculates the error between the calculated position and the target position `meta`, and prints the results.

```

1  clc;
2  clear;
3  close all;
4
5  q = [-5.09 -85.51 50.44 19.16 5.80 -45.70];
6  meta = [218.5 -76.3 285.7];
7
8  pos = fk(q);
9
10 error_x = abs(pos(1) - meta(1));
11 error_y = abs(pos(2) - meta(2));
12 error_z = abs(pos(3) - meta(3));
13 error_cartesiano = norm(pos - meta);
14
15 fprintf('q = [%2f %2f %2f %2f %2f %2f] grados\n', q);
16 fprintf('meta = [%2f %2f %2f] mm\n', meta);
17 fprintf('fk(q) = [%2f %2f %2f] mm\n', pos);
18 fprintf('error x = %2f mm\n', error_x);
19 fprintf('error y = %2f mm\n', error_y);
20 fprintf('error z = %2f mm\n', error_z);
21 fprintf('error cartesiano = %2f mm\n', error_cartesiano);
22
23 function pos = fk(q)
24     d1 = 134.75;
25     a2 = -110;
26     a3 = -96;
27     d4 = 63.4;
28     d5 = 75.05;
29     d6 = 50;
30
31     a1 = dh(q(1), d1, 0, 90);
32     a2_matriz = dh(q(2) - 90, 0, a2, 0);
33     a3_matriz = dh(q(3), 0, a3, 0);
34     a4 = dh(q(4) - 90, d4, 0, 90);
35     a5 = dh(q(5) + 90, d5, 0, -90);
36     a6 = dh(q(6), d6, 0, 0);

```

Console output:

```

q = [-5.09 -85.51 50.44 19.16 5.80 -45.70] grados
meta = [218.50 -76.30 285.70] mm
fk(q) = [227.14 -78.81 280.47] mm
error x = 8.64 mm
error y = 2.51 mm
error z = 5.23 mm
error cartesiano = 10.40 mm

```

Figura 4.2. Script de auditoría en MATLAB y salida por consola con el cálculo del error de posición cartesiana respecto a la meta.

4.5 Comparación y error medido

Los resultados de la auditoría, obtenidos al evaluar el vector articular alcanzado q en la función $fk(q)$, se resumen en la Tabla 4.2:

Tabla 4.2. Comparación entre la posición meta solicitada y la posición teórica predicha mediante $FK(q)$.

Magnitud	Solicitado (mm)	FK(q) (mm)	Error $ \Delta $ (mm)
x	218.50	227.14	8.64
y	-76.30	-78.81	2.51

Magnitud	Solicitado (mm)	FK(q) (mm)	Error $ \Delta $ (mm)
z	285.70	280.47	5.23
Error cartesiano (norma euclidiana e)	—	—	10.40

Por eje, los errores son de 8.64 mm en x, 2.51 mm en y y 5.23 mm en z; ninguno supera 10 mm de forma individual. Sin embargo, la norma euclidiana de estas diferencias da un error de posición global de 10.40 mm, que excede en 0.40 mm el valor de referencia de 10 mm indicado en las métricas del reto. El resultado se reporta tal como se obtuvo en la medición.

4.6 Análisis del error

Al descomponer el error de posición de 10.40 mm se observa lo siguiente:

- **Predominio del eje x:** el error en x (8.64 mm) es la componente principal, con cerca del 69 % de la suma de errores cuadráticos ($8.64^2 \approx 74.65 \text{ mm}^2$ frente a $e^2 \approx 108.16 \text{ mm}^2$). En términos prácticos, la FK ubica el efector unos 8.64 mm más adelante de lo solicitado en esa dirección.
- **Coherencia con el Ítem 1:** el error global es del mismo orden que las discrepancias observadas en el Ítem 1 (errores euclidianos entre 5.37 mm y 7.58 mm). Esto sugiere que una parte importante del error proviene de la diferencia entre la parametrización DH propia y la cinemática interna del firmware, y no de una falla de convergencia del solver de cinemática inversa.
- **Separación de fuentes de error:** para distinguir la imprecisión del solver de la discrepancia entre modelos (DH propio frente a firmware) habría que comparar la posición que reporta `get_coords()` inmediatamente después del movimiento con la meta y con la predicción $P_{FK}(q)$. Esta comprobación queda como trabajo complementario.

4.7 ¿Por qué eligió el brazo esta solución?

Con el vector ejecutado $q = [-5.09, -85.51, 50.44, 19.16, 5.80, -45.70]^\circ$ y la tabla DH, la FK da las siguientes posiciones intermedias respecto a la base (Tabla 4.3):

Tabla 4.3. Coordenadas tridimensionales de las articulaciones del JetCobot y posición calculada del efector final.

Punto	x (mm)	y (mm)	z (mm)
Hombro (origen del marco 1)	0.0	0.0	134.8
Codo (origen del marco 2)	109.2	-9.7	143.4
Muñeca (origen del marco 3)	164.2	-14.6	221.9
Efector final (FK)	227.1	-78.8	280.5

El brazo superior sale casi horizontal hacia adelante y el antebrazo sube, de modo que el codo queda por debajo de la línea que une el hombro con la muñeca. Existe otra solución, con el codo del lado contrario, que lleva el efector a la misma posición con una configuración articular distinta.

Hipótesis: el firmware elige, entre las soluciones válidas, la más cercana a la configuración articular de partida, lo que minimiza el movimiento de los servos y respeta los límites articulares. Bajo esta hipótesis, la configuración con el codo por debajo correspondería a la que exige un menor cambio articular desde la postura inicial; sin embargo, esa postura inicial no se registró, por lo que no puede confirmarse con los datos disponibles.

Esta explicación es una hipótesis razonable, pero no se comprobó experimentalmente. Para ponerla a prueba se enviaría la misma meta partiendo de una configuración inicial distinta (por ejemplo, con el codo flexionado hacia el otro lado): si el firmware devolviera una solución diferente, la hipótesis ganaría respaldo.

4.8 Conclusión

El firmware resolvió la cinemática inversa del objetivo (218.5, -76.3, 285.7) mm y la FK propia permitió auditar el resultado: los errores por eje fueron de 8.64, 2.51 y 5.23 mm, y el error cartesiano de 10.40 mm. Esta diferencia es del mismo orden que la observada entre el modelo DH propio y el firmware en el Ítem 1, por lo que la FK resulta consistente como instrumento de auditoría. La pregunta que abre la semana 5, es decir, por qué el brazo prefirió esta solución y no la del codo contrario, queda planteada con una hipótesis (cercanía a la configuración de partida) pendiente de verificación.

Cierre reflexivo — Política de cola para CapyTown

Tras analizar los resultados experimentales obtenidos en las mediciones bajo contención, la política de cola que recomendamos implementar en el sistema de transporte de CapyTown es la Política de Prioridad con Envejecimiento (Priority Queue with Aging).

Justificación de la decisión

- **Superación del bloqueo por inanición (Starvation):** aunque la política de prioridad estricta garantiza un tiempo de respuesta óptimo para las solicitudes críticas de la ciudad, presenta el riesgo de postergar indefinidamente los paquetes de baja prioridad. La incorporación del mecanismo de envejecimiento (aging) resuelve esta limitación al incrementar gradualmente la prioridad de las tareas en espera, garantizando un límite superior razonable en los tiempos de latencia para todas las cargas de trabajo.
- **Comportamiento frente a FIFO:** si bien la política FIFO es simple y equitativa en cuanto al orden de llegada, resulta ineficiente ante ráfagas simultáneas de datos o emergencias operativas, ya que no discrimina la relevancia del tráfico. En un entorno robótico y distribuido como el de CapyTown (bajo ROS 2), priorizar mensajes clave sin desatender las métricas globales es fundamental para asegurar la estabilidad del sistema.
- **Calidad de Servicio (QoS) y previsibilidad:** las pruebas cuantitativas demuestran que esta política mantiene una variabilidad contenida en el jitter y evita la degradación catastrófica del rendimiento cuando la red alcanza niveles altos de saturación, ofreciendo un balance idóneo entre equidad, reactividad y robustez.

Bibliografía de apoyo

- Craig, J. (2005). *Introduction to Robotics: Mechanics and Control*, 3.^a ed. Pearson. Capítulos 2-3 (transformaciones y cinemática directa).
- Siciliano, B. et al. (2009). *Robotics: Modelling, Planning and Control*. Springer. Capítulo 2.
- Macenski, S. et al. (2022). Robot Operating System 2: Design, architecture, and uses in the wild. *Science Robotics*.
- ROS 2 Humble Documentation — Actions, Executors and Callback Groups.
- Yahboom JetCobot / pymycobot — documentación oficial del fabricante.